ENSIAS

جامعة محمد الخامس بالرباط
Université Mohammed V de Rabat

Filière : Génie Logiciel

Rapport du Projet de Fin d'Année

Système Multi-Agents de Suivi de Santé

Soutenu le 23 Juin 2026 devant le jury composé de :

Pr. EL HAMLAOUI Mahmoud	**Encadrant**
Pr. BOUNABAT Bouchaib	**Examinateur**

Réalisé par :

REZGUANI Hiba

BAHBAH Ikram

Année Universitaire : 2025–2026

Remerciements

Au terme de ce travail, nous souhaitons exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'année. Leur soutien, leurs encouragements et leurs précieux conseils nous ont accompagnées tout au long de cette expérience enrichissante.

Nous tenons tout particulièrement à adresser nos sincères remerciements à **Monsieur le Professeur EL HAMLAOUI Mahmoud**, notre encadrant, pour sa disponibilité, son accompagnement constant, sa patience ainsi que la qualité de ses orientations. Son expertise, sa rigueur scientifique et ses conseils avisés ont constitué une aide précieuse durant toutes les phases de ce projet. Son encadrement nous a permis d'acquérir de nouvelles compétences et d'aborder ce travail avec méthode et professionnalisme.

Nous adressons également nos vifs remerciements à **Monsieur le Professeur BOUNABAT Bouchaib**, membre du jury, pour l'intérêt porté à notre travail ainsi que pour le temps consacré à son évaluation. Nous lui sommes reconnaissantes pour ses remarques constructives et ses recommandations qui contribueront à l'amélioration de nos futurs travaux.

Nous exprimons également notre sincère reconnaissance à l'ensemble du corps professoral de l'**École Nationale Supérieure de l'Informatique et d'Analyse des Systèmes (ENSIAS)** pour la qualité de l'enseignement dispensé, la richesse des connaissances transmises et l'excellence de la formation qui nous ont permis de mener à bien ce projet.

À ces guides bienveillants, nous exprimons notre profonde reconnaissance.

Sincèrement, **Mlle. Hiba REZGUANI & Mlle. Ikram BAHBAH**

Résumé

Ce projet porte sur la conception et la réalisation d'un système intelligent de suivi de santé basé sur une architecture multi-agents et l'Internet des Objets (IoT). L'objectif principal est d'améliorer l'assistance médicale et le suivi de santé en combinant un assistant médical intelligent capable de répondre aux questions des utilisateurs et un système de surveillance exploitant les données physiologiques provenant d'une montre connectée.

La solution développée repose sur plusieurs agents spécialisés collaborant pour analyser les requêtes médicales, interpréter les données de santé, détecter les anomalies et générer des réponses ou des alertes adaptées. Afin d'améliorer la pertinence des résultats, certains agents exploitent un mécanisme de génération augmentée par récupération (RAG) basé sur des ressources médicales de référence. Le système intègre également des mécanismes d'évaluation et de validation permettant d'assurer la qualité des informations produites.

La réalisation s'appuie sur une architecture moderne utilisant React.js pour l'interface utilisateur, FastAPI pour les services backend, CrewAI pour l'orchestration des agents intelligents, LangChain pour l'intégration des modèles de langage et Pinecone pour la recherche sémantique. Les données médicales et les informations utilisateur sont stockées dans une base de données SQLite.

Les résultats obtenus montrent que le système est capable de fournir des réponses médicales cohérentes dans différents domaines de la santé, d'assurer le routage des requêtes vers les agents appropriés, d'exploiter efficacement les connaissances documentaires via le mécanisme RAG et de détecter automatiquement certaines anomalies physiologiques à partir des données issues des objets connectés.

Mots-clés : Intelligence artificielle, Système multi-agents, Suivi de santé, Internet des Objets (IoT), Modèles de langage (LLM), RAG, Surveillance médicale, GPT-4o Mini, CrewAI.

Abstract

This project focuses on the design and implementation of an intelligent health monitoring system based on a multi-agent architecture and the Internet of Things (IoT). The main objective is to improve medical assistance and health monitoring by combining an intelligent medical assistant capable of answering users' health-related questions with a monitoring system that analyzes physiological data collected from a connected smartwatch.

The proposed solution relies on several specialized agents that collaborate to process medical queries, interpret health data, detect anomalies, and generate appropriate responses or alerts. To enhance the relevance and quality of the generated information, some agents integrate a Retrieval-Augmented Generation (RAG) mechanism supported by medical reference resources. The system also includes evaluation and validation mechanisms to ensure the consistency and quality of the produced results.

The implementation is based on a modern software architecture using React.js for the user interface, FastAPI for backend services, CrewAI for intelligent agent orchestration, LangChain for language model integration, and Pinecone for semantic search. Medical records and user information are stored in an SQLite database.

The obtained results demonstrate that the system is capable of providing coherent medical responses across different healthcare domains, correctly routing queries to the appropriate specialized agents, effectively exploiting external medical knowledge through the RAG mechanism, and automatically detecting physiological anomalies from data collected by connected devices.

Keywords : Artificial Intelligence, Multi-Agent System, Health Monitoring, Internet of Things (IoT), Large Language Models (LLM), Retrieval-Augmented Generation (RAG), Medical Monitoring, GPT-4o Mini, CrewAI.

Liste des abréviations

Table 1 – Liste des abréviations

Abréviation	**Désignation**
API	Application Programming Interface
AWS	Amazon Web Services
CBT	Cognitive Behavioural Therapy (Thérapie Cognitivo-Comportementale)
FC	Fréquence Cardiaque
GPT	Generative Pre-trained Transformer
HTTP	HyperText Transfer Protocol
IA	Intelligence Artificielle
IoT	Internet of Things (Internet des Objets)
JSON	JavaScript Object Notation
LLM	Large Language Model (Modèle de Langage de Grande Taille)
NHS	National Health Service
NIMH	National Institute of Mental Health
RAG	Retrieval-Augmented Generation (Génération Augmentée par Récupération)
REST	Representational State Transfer
SpO_2	Saturation périphérique en oxygène
SQL	Structured Query Language
UML	Unified Modeling Language
UI	User Interface (Interface Utilisateur)
URL	Uniform Resource Locator

Table des matières

Remerciements **2**

Résumé **3**

Abstract **4**

Liste des abréviations **5**

Introduction Générale **11**

1 Présentation et Cadrage du Projet **12**
Introduction . . . 12
1.1 Présentation du sujet . . . 12
1.2 Cahier des Charges . . . 12
1.2.1 Problématique . . . 12
1.2.2 Objectifs . . . 13
1.3 Étude de l'existant . . . 13
1.3.1 Amazon Health AI . . . 13
1.3.2 MedSync AI . . . 14
1.3.3 Apple Watch Vitals . . . 14
1.3.4 Comparaison des solutions existantes . . . 14
1.3.5 Produit du projet (Solution proposée) . . . 14
1.3.6 Besoins fonctionnels . . . 15
1.3.7 Besoins non fonctionnels . . . 15
1.4 Planification Opérationnelle . . . 15
1.4.1 Tableau des tâches . . . 15
1.4.2 Description des tâches . . . 16
1.4.2.1 Étude bibliographique . . . 16
1.4.2.2 Analyse des besoins . . . 16
1.4.2.3 Conception et modélisation UML . . . 16
1.4.2.4 Réalisation et implémentation du système intelligent de suivi de santé basé sur les agents et l'IoT . . . 16
1.4.2.5 Rédaction du rapport . . . 16
1.4.2.6 Préparation de la soutenance . . . 17
1.4.3 Diagramme de Gantt . . . 17
Conclusion . . . 17

2 Analyse et Conception **18**
Introduction . . . 18
2.1 Établissement du diagramme de cas d'utilisation . . . 18
2.2 Établissement du diagramme de classes . . . 19
2.2.1 Règles de gestion . . . 20
2.3 Établissement des diagrammes d'activité . . . 21
2.3.1 Diagramme d'activité du système de réponse utilisateur . . . 22
2.3.2 Diagramme d'activité du système SmartWatch IoT . . . 24
2.4 Établissement des diagrammes de séquence . . . 25
2.4.1 Diagramme de séquence du système de réponse utilisateur . . . 25
2.4.2 Diagramme de séquence du système SmartWatch IoT . . . 26
2.5 Établissement du diagramme de composants . . . 27
2.6 Établissement du diagramme de déploiement . . . 29
2.7 Description des agents . . . 31
2.7.1 Agent lecteur et routeur . . . 32
2.7.2 Agent de santé générale . . . 32
2.7.3 Agent santé mentale . . . 32
2.7.4 Agent pharmacie . . . 33
2.7.5 Agent nutrition . . . 33
2.7.6 Agent clinique . . . 33
2.7.7 Agent monitoring . . . 33
2.7.8 Agent évaluateur . . . 34
2.7.9 Agent rédacteur . . . 34
2.7.10 Agent manager . . . 34
Conclusion . . . 34

3 Réalisation et Résultats **35**
Introduction . . . 35
3.1 Environnement matériel . . . 35
3.2 Environnement logiciel . . . 36
3.2.1 Python . . . 36
3.2.2 React.js . . . 36
3.2.3 FastAPI . . . 37
3.2.4 CrewAI . . . 37
3.2.5 LangChain . . . 38
3.2.6 OpenAI GPT-4o Mini . . . 38
3.2.7 Pinecone . . . 39
3.2.8 SQLite . . . 40
3.3 Architecture de l'intelligence artificielle . . . 40
3.3.1 Création des agents intelligents . . . 40
3.3.2 Base de connaissances vectorielle . . . 42
3.4 Présentation de l'interface de l'application . . . 44
3.4.1 Interface de connexion . . . 44
3.4.2 Interface d'inscription . . . 44
3.4.3 Interface de récupération du mot de passe . . . 45
3.4.4 Interface d'accueil . . . 46
3.4.5 Assistant médical intelligent . . . 47
3.4.6 Interface de surveillance médicale IoT . . . 47

3.4.7 Système d'alerte médicale 48
3.5 Validation du système 49
3.5.1 Validation de la pertinence des réponses médicales 49
3.5.2 Validation du chemin de traitement des requêtes 51
3.5.3 Validation du système de surveillance IoT 52
3.5.4 Validation de l'apport du mécanisme RAG 52
3.5.4.1 Comparaison des réponses obtenues 53
Conclusion 54

Conclusion Générale et Perspectives **55**

Références **56**

Table des figures

1.1 Planification des tâches du projet 16
1.2 Logo du diagramme de Gantt 17
1.3 Diagramme de Gantt du projet 17

2.1 Diagramme de cas d'utilisation global du système 19
2.2 Diagramme de classes du modèle de données du système 20
2.3 Diagramme d'activité du système de réponse utilisateur 22
2.4 Diagramme d'activité du système SmartWatch IoT 24
2.5 Diagramme de séquence du système de réponse utilisateur 25
2.6 Diagramme de séquence du système SmartWatch IoT 26
2.7 Diagramme de composants du système multi-agents de suivi de santé . . . 28
2.8 Diagramme de déploiement du système multi-agents de suivi de santé . . . 30

3.1 Logo Python 36
3.2 Logo React.js 37
3.3 Logo FastAPI 37
3.4 Logo CrewAI 38
3.5 Logo LangChain 38
3.6 Logo OpenAI GPT-4o Mini 39
3.7 Logo Pinecone 40
3.8 Logo SQLite 40
3.9 Vue d'ensemble des index vectoriels dans la console Pinecone 43
3.10 Interface de connexion 44
3.11 Interface d'inscription 45
3.12 Interface de récupération du mot de passe 45
3.13 Courriel de réinitialisation du mot de passe 46
3.14 Interface de réinitialisation du mot de passe 46
3.15 Interface d'accueil 47
3.16 Interface de l'assistant médical intelligent 47
3.17 Interface de surveillance des données IoT 48
3.18 Exemple d'alerte médicale générée automatiquement 48
3.19 Réponse sans utilisation du mécanisme RAG 53
3.20 Réponse générée avec utilisation du mécanisme RAG 54

Liste des tableaux

1 Liste des abréviations 5

3.1 Configuration matérielle utilisée 35
3.2 Configuration des agents du système 41
3.3 Format de sortie de chaque agent 42
3.4 Caractéristiques des index Pinecone 43
3.5 Validation de la pertinence des réponses médicales 50
3.6 Validation du chemin de traitement des requêtes 51
3.7 Validation de la détection des anomalies IoT 52
3.8 Comparaison des réponses avec et sans RAG 53

Introduction Générale

Dans un contexte marqué par l'évolution rapide des technologies numériques et de l'intelligence artificielle, le domaine de la santé connaît une transformation profonde. Les systèmes intelligents, les modèles de langage de grande taille (LLM) et les objets connectés offrent aujourd'hui de nouvelles possibilités pour améliorer l'accès à l'information médicale, assister les utilisateurs dans leurs démarches de santé et assurer une surveillance continue de certains paramètres physiologiques.

Cependant, l'accès à des informations médicales fiables demeure un défi pour de nombreux utilisateurs. Les ressources disponibles sur Internet sont souvent dispersées, difficiles à interpréter et parfois peu adaptées au contexte spécifique de chaque situation. Par ailleurs, les dispositifs connectés génèrent quotidiennement une quantité importante de données physiologiques qui restent souvent sous-exploitées malgré leur potentiel pour le suivi de l'état de santé et la détection précoce d'anomalies.

Afin de répondre à ces problématiques, ce projet vise à concevoir et développer un système multi-agents de suivi de santé. La solution proposée combine un assistant médical intelligent capable de répondre aux questions des utilisateurs et un système de surveillance reposant sur l'analyse des données physiologiques provenant d'une montre connectée. Grâce à l'intégration des technologies d'intelligence artificielle, du mécanisme de génération augmentée par récupération (RAG) et des objets connectés, le système est en mesure de fournir des réponses contextualisées, d'analyser les constantes vitales et de générer des alertes en cas de situation nécessitant une attention particulière.

L'objectif principal de ce travail est de mettre en œuvre une solution capable d'améliorer l'assistance médicale et le suivi de santé tout en garantissant des informations de qualité et une expérience utilisateur simple et intuitive.

Pour présenter les différentes étapes de cette réalisation, ce rapport est structuré en trois chapitres. Le premier chapitre présente le cadrage du projet à travers la problématique, les objectifs, l'étude de l'existant, les besoins du système ainsi que la planification des travaux. Le deuxième chapitre est consacré à l'analyse et à la conception de la solution à l'aide des différents diagrammes UML et de la description de l'architecture proposée. Enfin, le troisième chapitre présente la réalisation du système, les technologies utilisées, l'architecture d'intelligence artificielle, les interfaces développées ainsi que les résultats obtenus lors de la phase de validation.

Ce travail met ainsi en évidence l'apport des architectures multi-agents, des modèles de langage et des technologies IoT dans le développement de solutions intelligentes dédiées à l'assistance médicale et au suivi de santé.

Chapitre 1

Présentation et Cadrage du Projet

Introduction

Ce chapitre présente le contexte général du projet ainsi que les principaux éléments ayant guidé sa réalisation. Il expose la problématique étudiée, les objectifs visés et l'étude des solutions existantes dans le domaine de la santé numérique. Il présente également la solution proposée, les besoins fonctionnels et non fonctionnels du système ainsi que la planification adoptée pour mener à bien le développement de la plateforme de suivi de santé basée sur une architecture multi-agents.

1.1 Présentation du sujet

L'évolution rapide des technologies numériques a profondément transformé le domaine de la santé. L'émergence de l'intelligence artificielle, des systèmes intelligents et des objets connectés offre aujourd'hui de nouvelles possibilités pour améliorer l'assistance médicale, la surveillance des patients et l'accès à l'information de santé.

Parallèlement, les utilisateurs recherchent de plus en plus des solutions capables de fournir des informations médicales et accessibles à tout moment. Par ailleurs, le développement des objets connectés a contribué à l'émergence de nouvelles approches permettant d'améliorer le suivi de la santé et d'assister les utilisateurs dans la gestion de leur bien-être au quotidien.

Dans ce contexte, notre projet consiste à développer un système intelligent de suivi de santé basé sur une architecture multi-agents. Cette solution vise à répondre à deux besoins principaux : l'assistance aux utilisateurs à travers des réponses médicales adaptées et la surveillance automatique des données physiologiques provenant d'objets connectés.

L'objectif est de mettre en place une plateforme capable d'analyser les informations reçues, d'identifier les situations nécessitant une attention particulière et de fournir des réponses ou des alertes afin d'améliorer le suivi de l'état de santé des utilisateurs.

1.2 Cahier des Charges

1.2.1 Problématique

L'accès à une information médicale pertinente constitue un défi majeur pour de nombreuses personnes. Les utilisateurs disposent aujourd'hui d'une quantité importante d'in-

formations disponibles sur Internet, mais ces informations sont souvent dispersées, difficiles à interpréter.

D'autre part, les objets connectés génèrent quotidiennement un volume important de données physiologiques qui restent souvent sous-exploitées. Pourtant, ces données peuvent fournir des indications précieuses sur l'état de santé d'un individu et permettre la détection précoce de certaines anomalies.

Face à cette situation, il devient nécessaire de disposer d'un système capable d'assister les utilisateurs dans leurs interrogations médicales tout en assurant une surveillance intelligente des données issues des capteurs connectés.

1.2.2 Objectifs

L'objectif principal du projet est de développer un système intelligent de suivi de santé reposant sur une approche multi-agents afin d'améliorer l'assistance médicale et la surveillance des constantes vitales.

Pour atteindre cet objectif général, plusieurs objectifs spécifiques ont été définis :

- Fournir des réponses médicales adaptées aux questions des utilisateurs.
- Faciliter l'accès à des informations médicales structurées et compréhensibles.
- Recevoir et traiter automatiquement les données physiologiques issues d'une montre connectée.
- Détecter les anomalies potentielles dans les constantes vitales surveillées.
- Générer des alertes lorsqu'une situation anormale est détectée.
- Améliorer la fiabilité des résultats grâce à des mécanismes de validation et de contrôle.
- Offrir une plateforme évolutive pouvant intégrer de nouvelles fonctionnalités à l'avenir.

1.3 Étude de l'existant

Avant de concevoir la solution proposée, une étude des systèmes existants dans le domaine de la santé numérique a été réalisée. Cette analyse a permis d'identifier les principales fonctionnalités offertes par les plateformes actuelles ainsi que les limites pouvant être adressées par notre système.

1.3.1 Amazon Health AI

Amazon Health AI est un assistant médical intelligent développé par Amazon dans le cadre de la plateforme One Medical. Ce système permet aux utilisateurs d'obtenir des informations de santé personnalisées, d'interpréter certains résultats médicaux, de gérer leurs traitements et d'être orientés vers des professionnels de santé lorsque cela est nécessaire.

Selon Amazon, Health AI repose sur une architecture multi-agents composée d'un agent principal chargé des interactions avec l'utilisateur, d'agents spécialisés dédiés à des tâches particulières, d'agents d'audit assurant le contrôle des échanges ainsi que d'agents de surveillance garantissant le bon fonctionnement du système. Cette organisation permet d'améliorer la qualité des réponses et la sécurité des interactions médicales.

1.3.2 MedSync AI

MedSync AI est une plateforme d'assistance médicale basée sur l'intelligence artificielle permettant aux utilisateurs d'évaluer leurs symptômes et d'obtenir des informations de santé personnalisées. Le système utilise une interface conversationnelle capable de poser des questions ciblées afin de mieux comprendre la situation médicale de l'utilisateur.

1.3.3 Apple Watch Vitals

Apple Watch Vitals est une fonctionnalité intégrée aux montres connectées Apple Watch permettant de surveiller plusieurs indicateurs physiologiques tels que la fréquence cardiaque, la fréquence respiratoire, la température du poignet ou encore la durée du sommeil.

Le système analyse périodiquement ces données afin d'identifier d'éventuelles variations inhabituelles et d'informer l'utilisateur lorsqu'un changement significatif est détecté. Cette solution contribue ainsi au suivi quotidien de l'état de santé à travers l'exploitation des données collectées par les capteurs embarqués.

1.3.4 Comparaison des solutions existantes

L'étude des solutions existantes montre que chaque plateforme répond à un besoin spécifique du domaine de la santé numérique. Amazon Health AI et MedSync AI offrent une assistance médicale intelligente basée sur l'analyse des symptômes et les interactions conversationnelles. Cependant, ces solutions ne permettent pas une surveillance continue des constantes vitales à partir d'objets connectés.

De son côté, Apple Watch Vitals assure un suivi efficace des données physiologiques et la détection de certaines variations anormales. Toutefois, cette solution ne fournit pas d'assistance médicale intelligente permettant d'interagir avec l'utilisateur ou d'interpréter les situations détectées.

Ainsi, les solutions étudiées se concentrent généralement soit sur l'assistance médicale, soit sur la surveillance des données physiologiques. Elles ne combinent pas simultanément une architecture multi-agents, une assistance médicale intelligente et une surveillance périodique des constantes vitales issues d'une montre connectée.

1.3.5 Produit du projet (Solution proposée)

La solution proposée consiste en une plateforme intelligente de suivi de santé intégrant des mécanismes d'assistance médicale et de surveillance de l'état de santé. Le système est conçu pour répondre à deux scénarios principaux :

- Le premier scénario concerne les utilisateurs souhaitant obtenir des informations ou des conseils relatifs à la santé. La plateforme traite les questions soumises et fournit des réponses adaptées en fonction du contexte médical identifié.
- Le second scénario concerne la surveillance des données provenant d'une montre connectée. Les informations reçues sont exploitées afin d'identifier d'éventuelles situations nécessitant une attention particulière.

Grâce à cette double approche, le système contribue à améliorer l'expérience utilisateur tout en favorisant un suivi plus efficace de l'état de santé.

1.3.6 Besoins fonctionnels

Afin de répondre aux objectifs définis, le système doit offrir les fonctionnalités suivantes :

- Permettre à l'utilisateur de soumettre des questions relatives à la santé.
- Générer automatiquement des réponses adaptées aux questions reçues.
- Recevoir les données physiologiques provenant d'une montre connectée.
- Enregistrer les mesures collectées dans une base de données.
- Analyser automatiquement les constantes vitales reçues.
- Identifier les situations normales et anormales.
- Générer des alertes en cas de détection d'anomalies.
- Permettre la consultation de l'historique des mesures enregistrées.
- Afficher les données sous forme de graphiques et d'indicateurs visuels.
- Fournir un suivi continu de l'évolution des constantes vitales.

1.3.7 Besoins non fonctionnels

Au-delà des fonctionnalités attendues, le système doit satisfaire plusieurs exigences de qualité.

- **Performance :** le système doit fournir les résultats dans un délai raisonnable afin d'assurer une bonne expérience utilisateur.
- **Fiabilité :** les analyses et les alertes générées doivent être cohérentes et pertinentes.
- **Sécurité :** les données médicales et personnelles des utilisateurs doivent être protégées contre tout accès non autorisé.
- **Maintenabilité :** le système doit être conçu de manière à faciliter les futures modifications et évolutions.
- **Scalabilité :** l'architecture doit permettre l'ajout de nouveaux modules ou fonctionnalités sans remise en cause de l'existant.
- **Ergonomie :** l'interface utilisateur doit être simple, intuitive et facile à utiliser.

1.4 Planification Opérationnelle

La réalisation du projet a été organisée selon une démarche progressive permettant de structurer les différentes phases de développement et d'assurer un suivi rigoureux de l'avancement des travaux. Cette planification a permis de répartir les tâches sur toute la durée du projet, depuis l'étude préliminaire jusqu'à la préparation de la soutenance.

1.4.1 Tableau des tâches

Le tableau présente les principales tâches réalisées durant le projet ainsi que leur période d'exécution.



Nom	Date de début	Date de fin
Étude bibliographique	12/03/2026	20/03/2026
Analyse des besoins	23/03/2026	27/03/2026
Conception et modélisation UML	30/03/2026	10/04/2026
Réalisation et implémentation du système intelligent de suivi de santé basé sur les agents et l'IoT	13/04/2026	12/06/2026
Rédaction du rapport	12/06/2026	19/06/2026
Préparation de la soutenance	18/06/2026	22/06/2026

Figure 1.1 – Planification des tâches du projet

1.4.2 Description des tâches

Les différentes tâches présentées dans le tableau précédent sont décrites comme suit :

1.4.2.1 Étude bibliographique

Cette étape a consisté à effectuer une recherche documentaire approfondie sur les concepts liés au projet, notamment les systèmes multi-agents, les modèles de langage (LLM), le mécanisme de génération augmentée par récupération (RAG), les bases de données vectorielles ainsi que les applications de l'Internet des Objets dans le domaine de la santé. Cette phase a permis d'acquérir les connaissances nécessaires à la conception de la solution proposée.

1.4.2.2 Analyse des besoins

Cette phase a été consacrée à l'identification des besoins fonctionnels et non fonctionnels du système. Elle a permis de définir les fonctionnalités attendues, les contraintes techniques ainsi que les différents scénarios d'utilisation de la plateforme.

1.4.2.3 Conception et modélisation UML

Cette étape a consisté à concevoir l'architecture globale du système et à modéliser son fonctionnement à travers différents diagrammes UML. Les diagrammes de cas d'utilisation, d'activité et de séquence ont permis de représenter les interactions entre les acteurs, les agents intelligents et les différents composants de l'application.

1.4.2.4 Réalisation et implémentation du système intelligent de suivi de santé basé sur les agents et l'IoT

Cette phase représente la partie principale du projet. Elle comprend le développement de l'interface utilisateur, l'implémentation du backend, l'intégration de l'architecture multi-agents, la mise en œuvre du mécanisme RAG, la création de la base documentaire médicale, l'intégration des objets connectés ainsi que le développement des fonctionnalités de surveillance et de détection automatique des anomalies médicales.

1.4.2.5 Rédaction du rapport

Cette étape a consisté à documenter l'ensemble du travail réalisé. Elle comprend la rédaction des différentes parties du rapport, notamment la présentation du projet, l'analyse et la conception, la réalisation technique ainsi que les résultats obtenus.

1.4.2.6 Préparation de la soutenance

La dernière phase du projet a été consacrée à la préparation de la soutenance. Elle comprend la réalisation du support de présentation, la préparation de la démonstration de l'application ainsi que l'organisation des différents éléments nécessaires à la présentation finale devant le jury.

1.4.3 Diagramme de Gantt

Figure 1.2 – Logo du diagramme de Gantt

Le diagramme de Gantt est un outil de gestion de projet permettant de représenter graphiquement les différentes tâches à réaliser ainsi que leur répartition dans le temps. Il offre une vision globale du projet en mettant en évidence les dates de début et de fin des activités, leur durée et leur enchaînement chronologique. Cet outil facilite la planification, le suivi de l'avancement des travaux et la coordination des différentes phases du projet.

Pour réussir ce projet, nous avons suivi le calendrier ci-dessous :

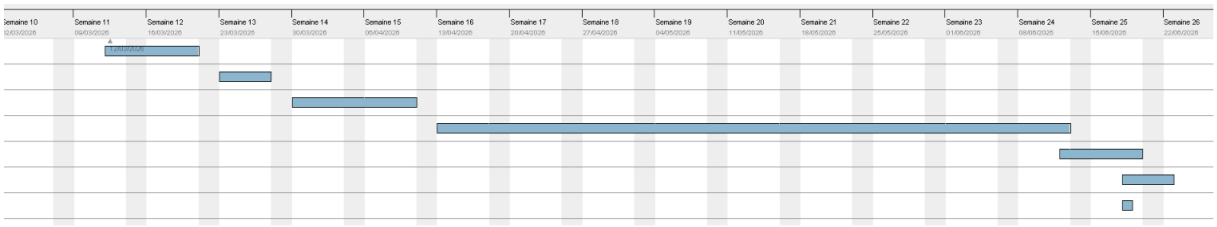


Figure 1.3 – Diagramme de Gantt du projet

Conclusion

Ce chapitre a permis de présenter le cadre général du projet à travers l'étude de la problématique, la définition des objectifs et l'analyse de quelques solutions existantes dans le domaine de la santé numérique. Il a également présenté la solution proposée ainsi que les besoins fonctionnels et non fonctionnels auxquels le système doit répondre. Enfin, la planification des différentes phases du projet a été établie afin d'assurer un développement structuré et cohérent. Cette étude constitue une base essentielle pour les étapes suivantes consacrées à l'analyse, à la conception et à la réalisation du système.

Chapitre 2

Analyse et Conception

Introduction

Ce chapitre est consacré à l'analyse et à la conception du système multi-agents de suivi de santé. Il présente les différents modèles UML utilisés pour décrire les fonctionnalités du système, sa structure de données ainsi que son architecture logicielle. Les diagrammes de cas d'utilisation, de classes, d'activité, de séquence, de composants et de déploiement permettent de représenter les différents aspects de la solution proposée. Enfin, les agents intelligents composant l'architecture multi-agents sont décrits afin de préciser leurs rôles et leurs responsabilités au sein du système.

2.1 Établissement du diagramme de cas d'utilisation

Le diagramme de cas d'utilisation présente les principales interactions entre l'utilisateur et le système multi-agents de suivi de santé. Afin d'accéder aux différentes fonctionnalités de la plateforme, l'utilisateur doit d'abord s'authentifier. Une fois connecté, il peut poser des questions médicales, consulter les réponses générées par les agents intelligents, accéder à l'historique des données, visualiser ses informations de santé ainsi que recevoir des alertes en cas d'anomalie détectée.

La figure suivante présente le diagramme de cas d'utilisation global du système et illustre les principales interactions entre l'utilisateur et la plateforme.

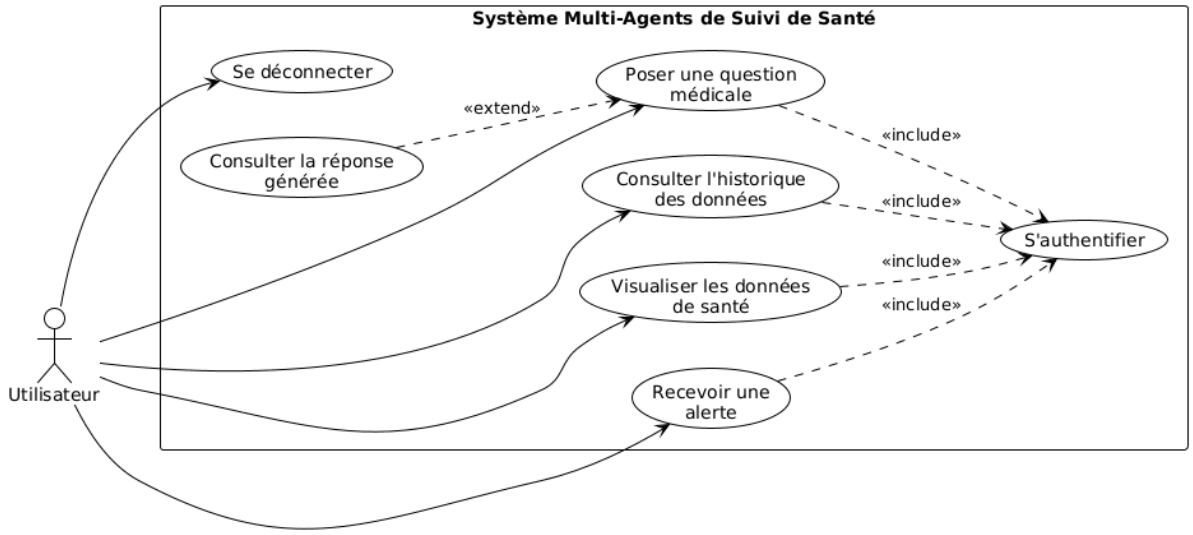


Figure 2.1 – Diagramme de cas d'utilisation global du système

2.2 Établissement du diagramme de classes

Le diagramme de classes permet de représenter la structure statique du système. Il met en évidence les principales entités manipulées par l'application, leurs attributs ainsi que les relations existant entre elles.

Dans notre système, le modèle de données repose principalement sur les entités liées à l'utilisateur, aux conversations, aux messages, aux mesures IoT et à la réinitialisation du mot de passe. Ce diagramme permet ainsi de visualiser l'organisation des données utilisées par la plateforme de suivi de santé.

La figure 2.2 présente le diagramme de classes du modèle de données du système.

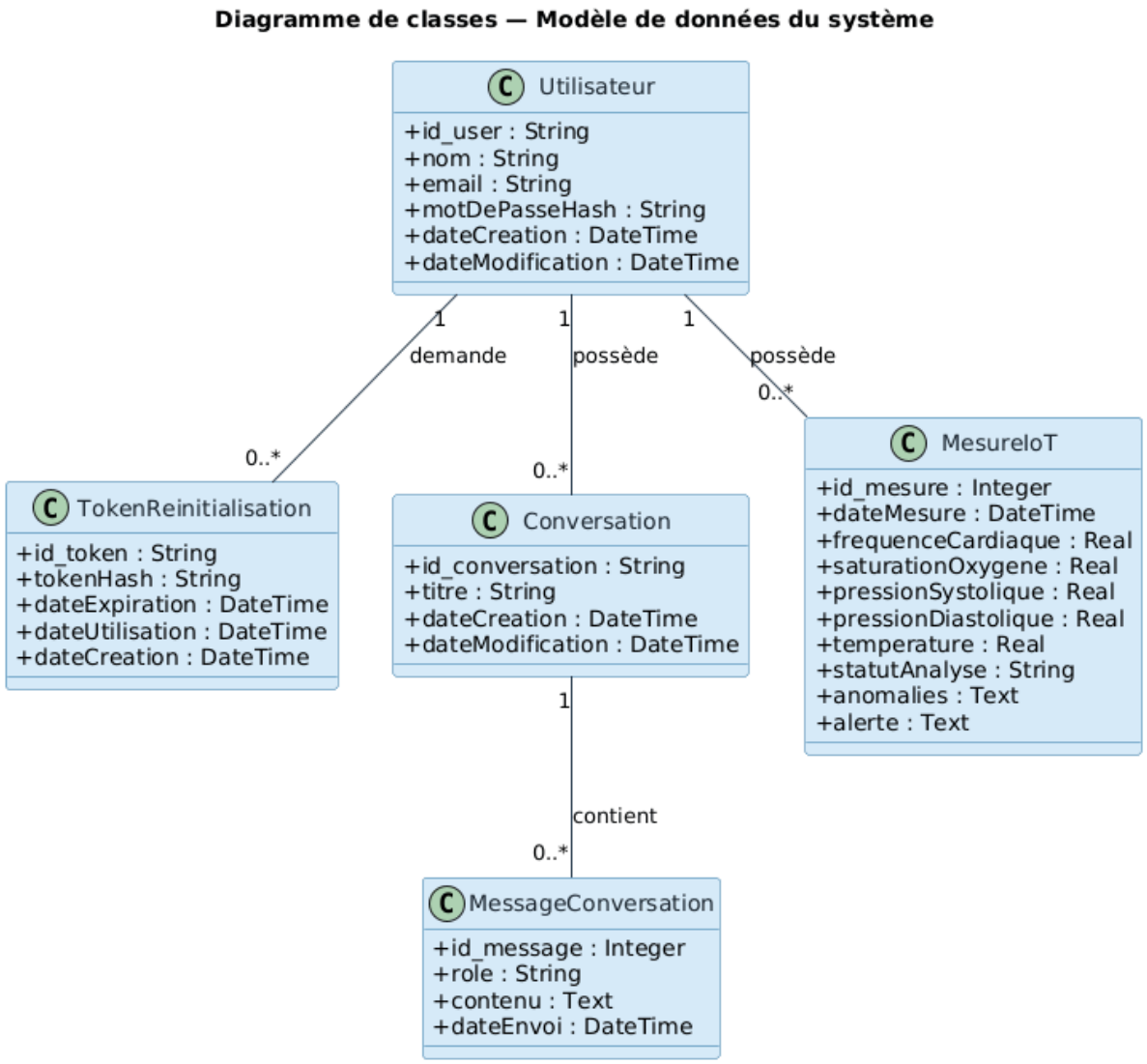


Figure 2.2 – Diagramme de classes du modèle de données du système

2.2.1 Règles de gestion

Les règles de gestion permettent de préciser les contraintes et les relations entre les différentes classes du système.

- **Relation Utilisateur – Conversation :** Un utilisateur peut posséder plusieurs conversations, tandis qu'une conversation appartient à un seul utilisateur. Cette relation permet d'associer chaque échange avec l'assistant médical à l'utilisateur concerné.
- **Relation Conversation – MessageConversation :** Une conversation peut contenir plusieurs messages, tandis que chaque message appartient à une seule conversation. Cette relation permet de conserver l'historique détaillé des échanges entre l'utilisateur et l'assistant médical.
- **Relation Utilisateur – TokenReinitialisation :** Un utilisateur peut générer plusieurs tokens de réinitialisation de mot de passe, tandis que chaque token est associé à un seul utilisateur. Cette relation permet de gérer les demandes de récupération du mot de passe de manière sécurisée.

- **Relation Utilisateur – MesureIoT :** Un utilisateur peut posséder plusieurs mesures IoT, tandis que chaque mesure est associée à un seul utilisateur. Cette relation permet de relier les constantes vitales collectées par la montre connectée au patient authentifié.

2.3 Établissement des diagrammes d'activité

Les diagrammes d'activité permettent de représenter les étapes de fonctionnement du système selon les deux scénarios principaux : le traitement d'une question utilisateur et le suivi des données issues d'une montre connectée.

2.3.1 Diagramme d'activité du système de réponse utilisateur

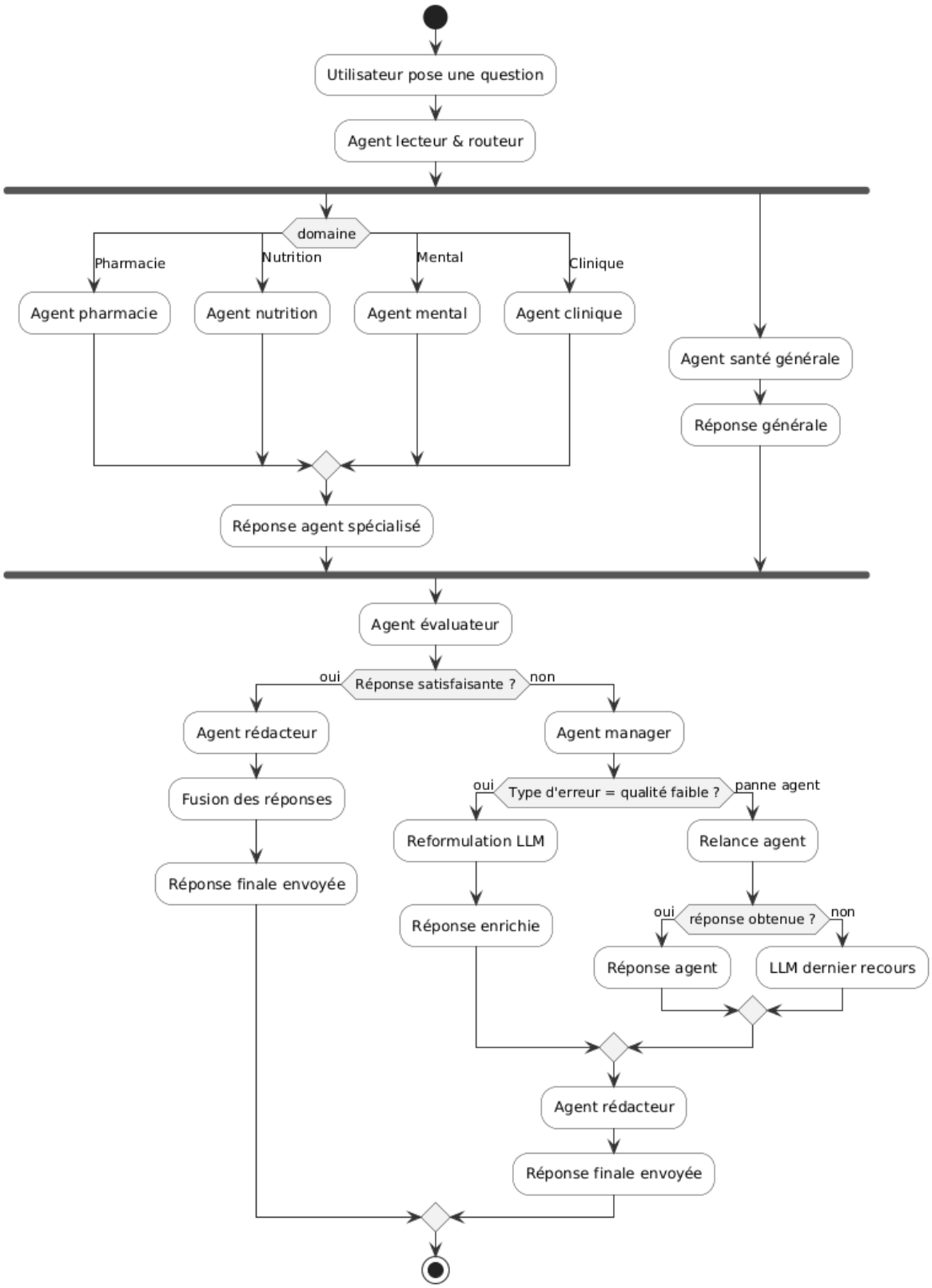


Figure 2.3 – Diagramme d'activité du système de réponse utilisateur

La figure 2.3 présente le déroulement du traitement d'une question soumise par l'utilisateur au sein du système multi-agents. Celui-ci repose sur une architecture organisée en plusieurs étapes successives de traitement, d'évaluation et de validation.

Lorsqu'un utilisateur soumet une question, celle-ci est d'abord prise en charge par un agent lecteur et routeur. Cet agent analyse la requête afin d'identifier le domaine médical concerné, qu'il s'agisse de la pharmacie, de la nutrition, de la santé mentale ou du domaine clinique.

Une fois le domaine identifié, le système déclenche une exécution parallèle permettant de lancer simultanément deux traitements. D'une part, l'agent spécialisé correspondant au domaine détecté produit une réponse ciblée. D'autre part, l'agent de santé générale fournit une analyse complémentaire basée sur sa base de connaissances. Cette approche permet de combiner une expertise spécifique avec une vision plus globale de la problématique.

Les réponses générées sont ensuite transmises à un agent évaluateur dont le rôle consiste à vérifier leur qualité, leur cohérence et leur pertinence.

Deux scénarios peuvent alors se présenter :

- Si la réponse est jugée satisfaisante, elle est transmise à l'agent rédacteur qui assure la fusion et la structuration des différentes informations afin de produire une réponse finale destinée à l'utilisateur.
- Si la réponse est jugée non satisfaisante, elle est transmise à l'agent manager. Celui-ci analyse l'origine du problème et met en œuvre les actions correctives nécessaires. En cas de problème de qualité, une reformulation peut être effectuée afin d'améliorer la réponse. En cas de défaillance d'un agent, une relance est réalisée et, si nécessaire, un mécanisme de secours est utilisé pour générer une réponse alternative.

Après correction ou amélioration, les résultats sont de nouveau évalués afin de garantir leur qualité. Une fois validée, la réponse finale est transmise à l'utilisateur.

2.3.2 Diagramme d'activité du système SmartWatch IoT

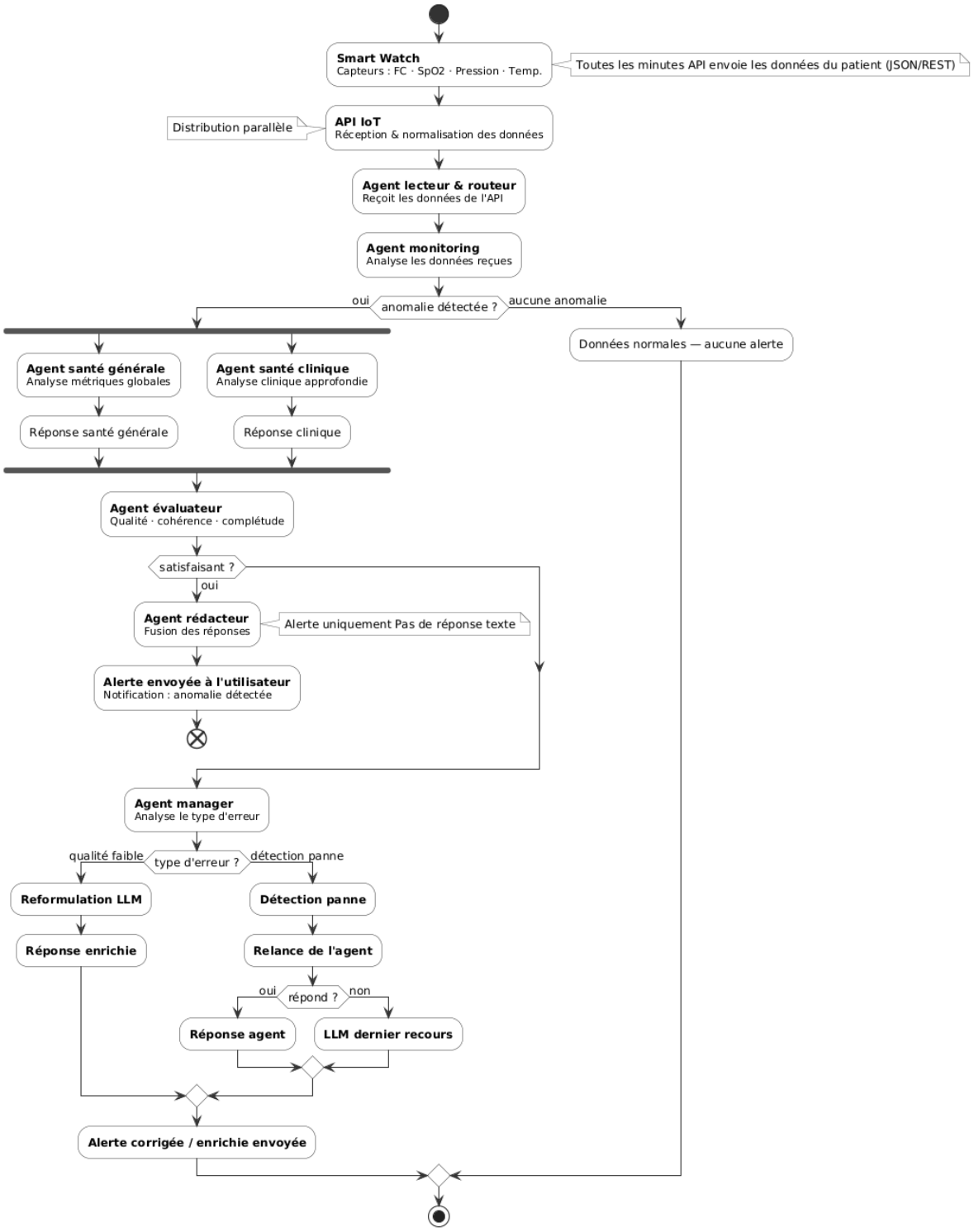


Figure 2.4 – Diagramme d'activité du système SmartWatch IoT

La figure 2.4 présente le déroulement du traitement des données provenant d'une montre connectée. Le processus débute par l'envoi périodique des données de santé vers la plateforme à travers une API dédiée. Après leur réception et leur normalisation, les

données sont transmises à un agent lecteur et routeur qui les dirige vers l'agent de monitoring.

L'agent de monitoring analyse les données reçues afin de déterminer si une anomalie est présente. Deux scénarios peuvent alors se produire. Dans le premier cas, aucune anomalie n'est détectée. Les données sont alors considérées comme normales et aucune alerte n'est générée. Dans le second cas, lorsqu'une anomalie est identifiée, le système déclenche un traitement multi-agents.

Dans ce scénario, les données sont analysées simultanément par un agent de santé générale et un agent de santé clinique. Les résultats obtenus sont ensuite transmis à un agent évaluateur chargé de vérifier leur qualité, leur cohérence et leur pertinence.

Si les analyses sont jugées satisfaisantes, elles sont transmises à l'agent rédacteur qui fusionne les résultats afin de générer une alerte destinée à l'utilisateur. En revanche, si une anomalie est détectée dans les réponses produites, un agent manager intervient afin d'identifier la source du problème. Selon le type d'erreur rencontré, le système peut effectuer une reformulation, relancer l'agent concerné ou faire appel à un mécanisme de secours.

Après correction et validation, une alerte enrichie est générée et transmise à l'utilisateur. Cette approche permet d'assurer un suivi continu de l'état de santé tout en garantissant la qualité et la fiabilité des alertes produites.

2.4 Établissement des diagrammes de séquence

Les diagrammes de séquence présentent les échanges chronologiques entre les acteurs et les composants du système.

2.4.1 Diagramme de séquence du système de réponse utilisateur

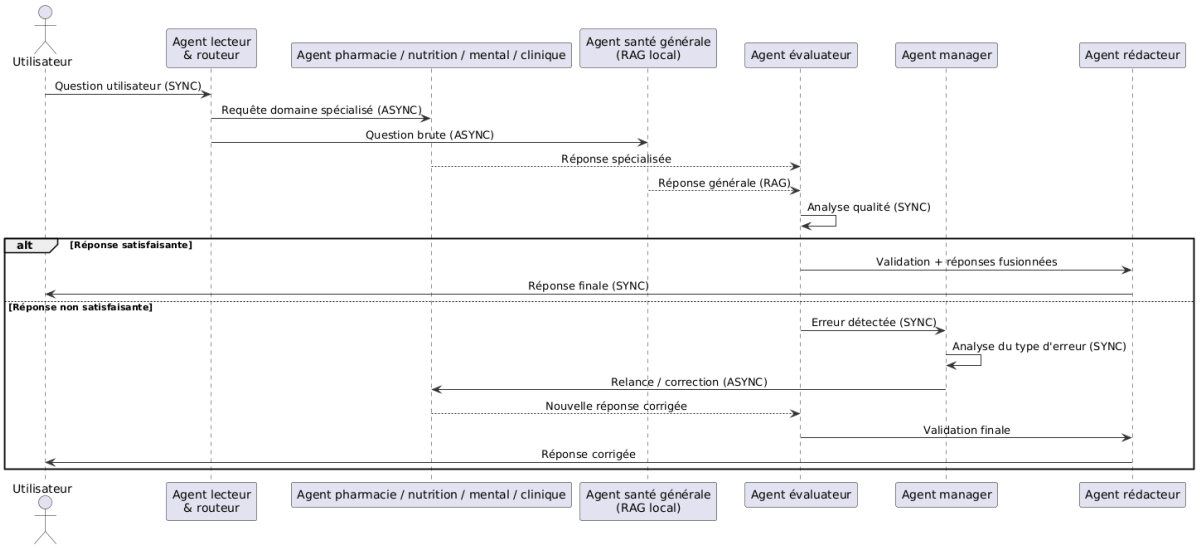


Figure 2.5 – Diagramme de séquence du système de réponse utilisateur

La figure 2.5 illustre le flux des messages échangés entre les différents composants du système lors du traitement d'une question utilisateur. Elle met en évidence les interactions

entre les agents impliqués dans l'analyse de la requête, l'évaluation des résultats et la génération de la réponse finale.

Le système repose sur une architecture hybride combinant des communications synchrones et asynchrones. Dans un premier temps, la question soumise par l'utilisateur est reçue par l'agent lecteur et routeur, qui identifie le domaine médical concerné et distribue la requête aux agents appropriés.

Les communications asynchrones interviennent principalement lors de la phase d'analyse. L'agent spécialisé correspondant au domaine identifié ainsi que l'agent de santé générale traitent simultanément la requête afin de produire des analyses complémentaires. Cette exécution parallèle permet d'améliorer les performances du système tout en enrichissant la qualité des informations obtenues.

Les résultats générés sont ensuite transmis à l'agent évaluateur. À partir de cette étape, les communications deviennent synchrones afin d'assurer un contrôle rigoureux du processus. L'agent évaluateur vérifie la qualité, la cohérence et la pertinence des réponses produites avant de prendre une décision.

Si les résultats sont jugés satisfaisants, ils sont transmis à l'agent rédacteur qui effectue la fusion des différentes analyses et génère la réponse finale destinée à l'utilisateur. Dans le cas contraire, l'agent évaluateur signale l'erreur à l'agent manager. Celui-ci analyse la nature du problème et déclenche les actions correctives nécessaires, telles qu'une relance des agents concernés ou une correction des réponses produites. Les nouvelles réponses obtenues sont ensuite soumises à une nouvelle phase de validation avant la génération de la réponse finale.

Ainsi, ce diagramme met en évidence la coordination entre les différents agents ainsi que l'ordre chronologique des échanges de messages. Il montre comment le système combine la rapidité du traitement parallèle avec la rigueur des mécanismes d'évaluation et de validation afin de garantir la qualité des réponses fournies à l'utilisateur.

2.4.2 Diagramme de séquence du système SmartWatch IoT

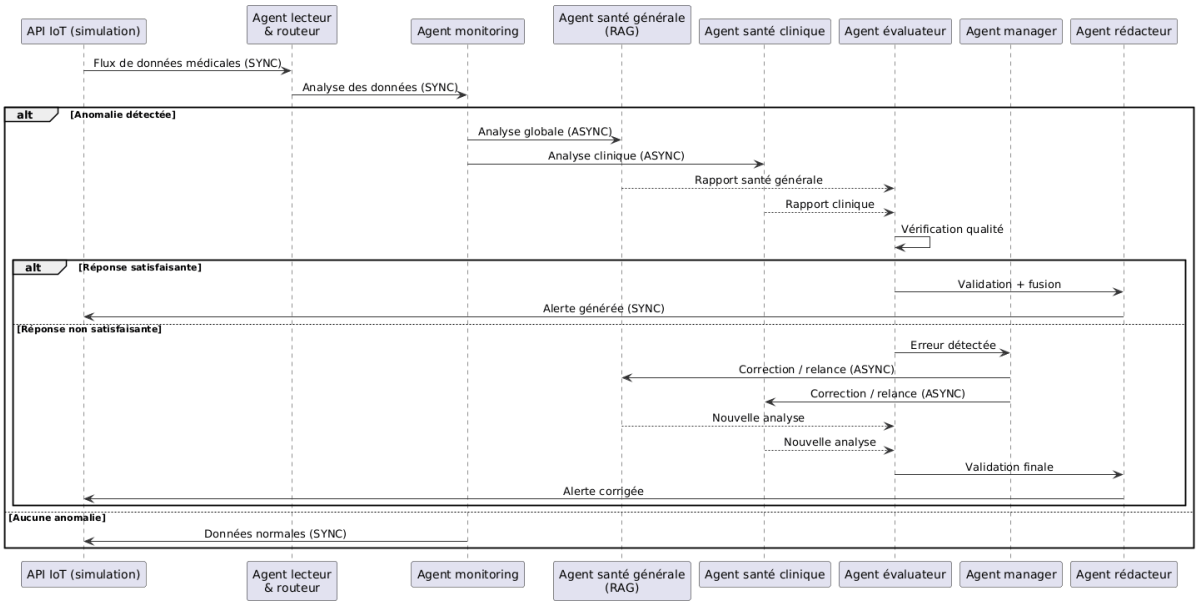


Figure 2.6 – Diagramme de séquence du système SmartWatch IoT

La figure 2.6 illustre le flux des messages échangés entre les différents composants du système lors du traitement des données provenant de la montre connectée. Elle met en évidence les interactions entre les agents impliqués dans l'analyse des données, la détection d'anomalies et la génération d'éventuelles alertes.

Le système repose sur une architecture hybride combinant des communications synchrones et asynchrones. Dans un premier temps, les données transmises par l'API IoT sont reçues et analysées par l'agent de monitoring. Cette phase suit un échange synchrone permettant de garantir la bonne transmission des informations entre les différents composants du système.

Lorsqu'une anomalie est détectée, le système déclenche des traitements asynchrones. L'agent de santé générale et l'agent de santé clinique réalisent alors leurs analyses en parallèle afin de produire rapidement des évaluations complémentaires de la situation. Cette exécution simultanée permet d'améliorer les performances du système tout en enrichissant la qualité des résultats obtenus.

Les phases d'évaluation, de validation, de correction et de génération d'alerte reposent quant à elles sur des communications synchrones. Chaque étape dépend du résultat de l'étape précédente afin de garantir la cohérence et la fiabilité des analyses produites. En cas de résultat non satisfaisant, des mécanismes de correction ou de relance peuvent être déclenchés avant une nouvelle validation.

Ainsi, ce diagramme met en évidence la coordination entre les différents agents ainsi que l'ordre chronologique des échanges de messages. Il montre comment le système combine la rapidité du traitement parallèle avec la rigueur des mécanismes de contrôle afin de produire des alertes fiables et pertinentes.

2.5 Établissement du diagramme de composants

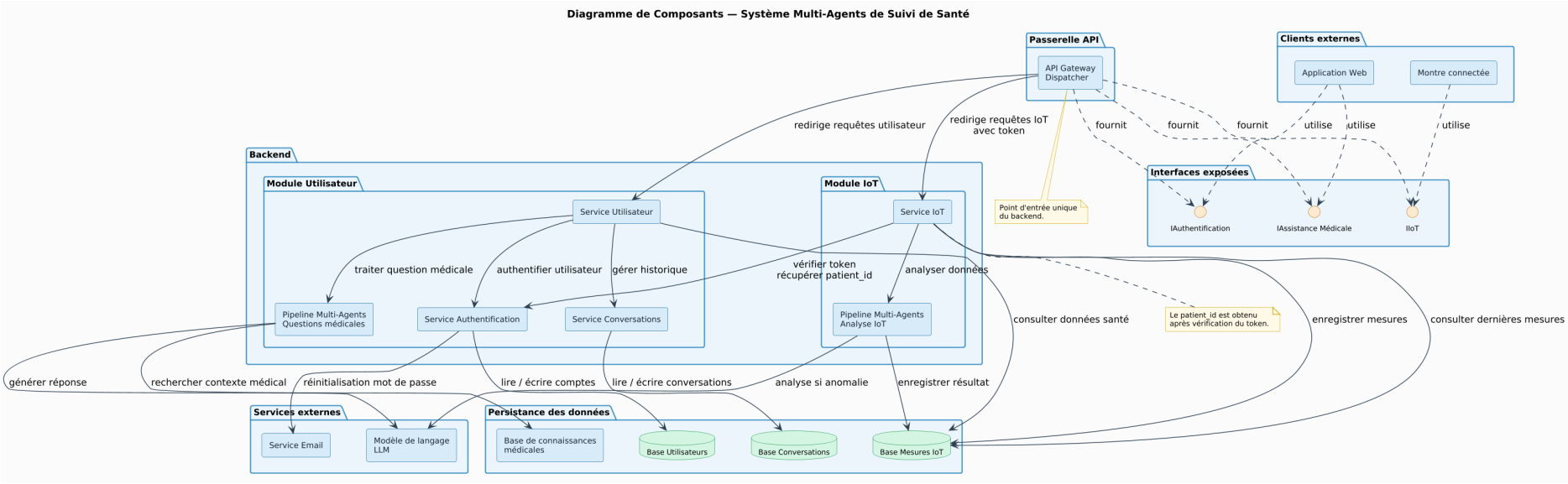


Figure 2.7 – Diagramme de composants du système multi-agents de suivi de santé

Le diagramme de composants permet de représenter l'organisation logicielle du système multi-agents de suivi de santé. Il met en évidence les principaux modules de la plateforme, leurs responsabilités ainsi que les dépendances entre les différents composants logiciels.

La figure 2.7 présente l'architecture globale du système. Celui-ci est composé de clients externes, d'une passerelle API, d'un backend, d'une couche de persistance des données et de services externes.

Les clients externes regroupent l'application web utilisée par l'utilisateur ainsi que la montre connectée chargée d'envoyer les mesures physiologiques. Ces clients accèdent au système à travers des interfaces exposées, notamment l'interface d'authentification, l'interface d'assistance médicale et l'interface IoT.

La passerelle API joue le rôle de point d'entrée unique du backend. Elle reçoit les requêtes provenant de l'application web et de la montre connectée, puis les redirige vers les services appropriés. Les requêtes liées à l'utilisateur sont transmises au module utilisateur, tandis que les données issues de la montre connectée sont orientées vers le module IoT.

Le module utilisateur regroupe les composants responsables de l'authentification, de la gestion des conversations et du traitement des questions médicales. Le service utilisateur communique avec le service d'authentification pour vérifier l'identité de l'utilisateur, avec le service conversations pour gérer l'historique, et avec le pipeline multi-agents afin de traiter les questions médicales. Ce pipeline exploite une base de connaissances médicale et un modèle de langage afin de générer des réponses adaptées.

Le module IoT est chargé de la réception et de l'analyse des données physiologiques. Le service IoT vérifie le token de l'utilisateur, récupère l'identifiant du patient, enregistre les mesures reçues dans la base dédiée, puis déclenche le pipeline multi-agents d'analyse IoT. Ce pipeline analyse les données afin d'identifier d'éventuelles anomalies et enregistre le résultat de l'analyse.

La couche de persistance regroupe les bases de données nécessaires au fonctionnement du système, notamment la base des utilisateurs, la base des conversations, la base des mesures IoT ainsi que la base de connaissances médicales. Enfin, les services externes comprennent le modèle de langage utilisé pour la génération et l'analyse des réponses, ainsi que le service email employé pour la réinitialisation du mot de passe.

Ainsi, ce diagramme permet de visualiser clairement la séparation des responsabilités entre les différents composants logiciels du système et montre comment ces composants collaborent pour assurer à la fois l'assistance médicale intelligente et le suivi des données de santé.

2.6 Établissement du diagramme de déploiement

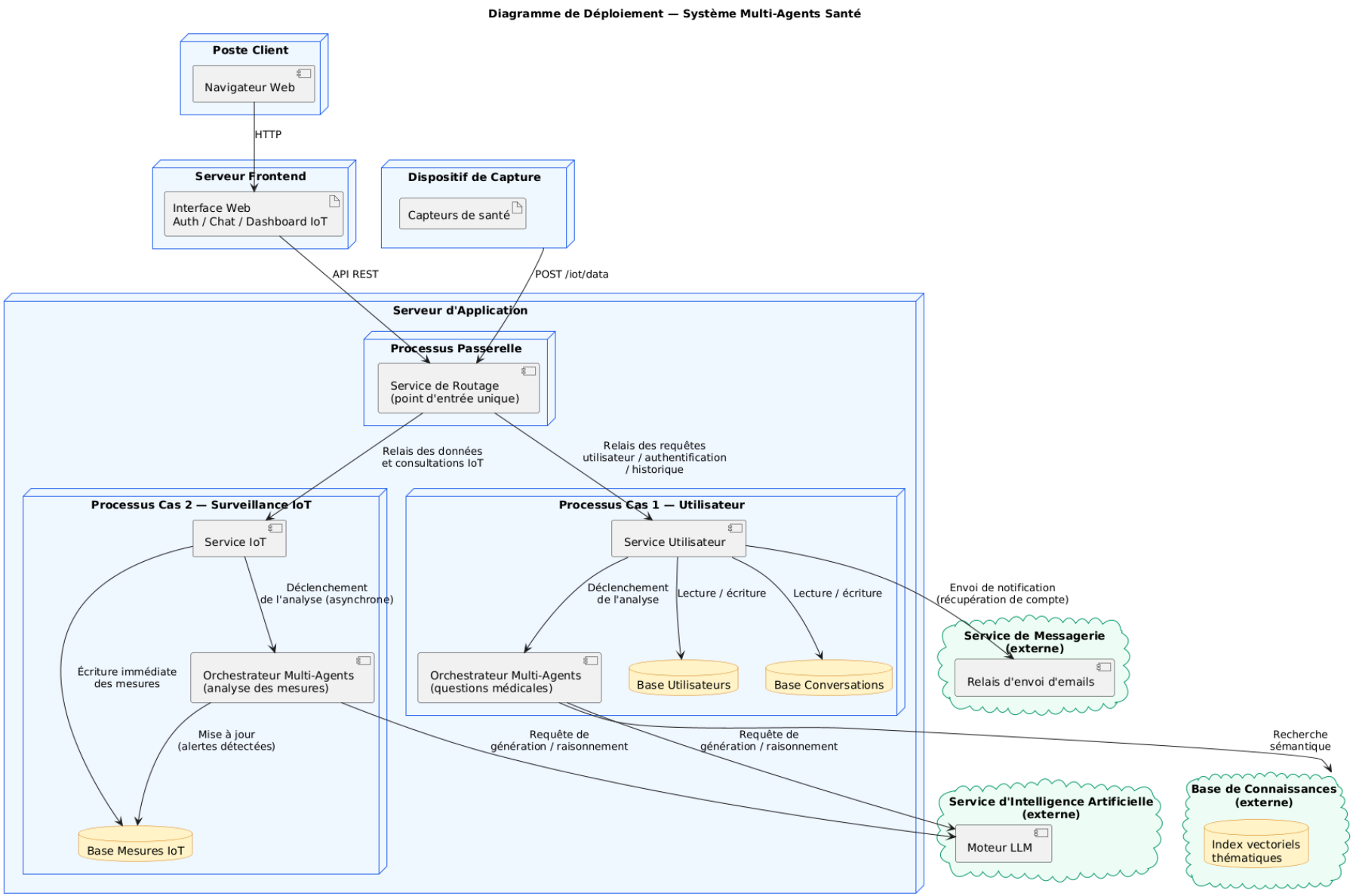


FIGURE 2.8 – Diagramme de déploiement du système multi-agents de suivi de santé

Le diagramme de déploiement permet de représenter l'architecture physique du système multi-agents de suivi de santé ainsi que la répartition des composants logiciels sur les différents nœuds d'exécution.

La figure 2.8 illustre l'environnement de déploiement du système. Celui-ci est constitué d'un poste client, d'un serveur frontend, d'un dispositif de capture des données physiologiques, d'un serveur d'application et de plusieurs services externes.

Le poste client correspond à l'équipement utilisé par l'utilisateur pour accéder à la plateforme à travers un navigateur web. Ce navigateur communique avec le serveur frontend via le protocole HTTP. Le serveur frontend héberge l'interface web permettant l'authentification, l'assistance médicale conversationnelle ainsi que la consultation des données de santé.

Les données physiologiques sont collectées par une montre connectée équipée de capteurs de santé. Ce dispositif transmet les mesures au serveur d'application à travers l'API IoT du système.

Le serveur d'application constitue le cœur de la plateforme. Il héberge trois processus distincts. Le premier correspond au processus passerelle, qui joue le rôle de point d'entrée unique et assure le routage des requêtes vers les différents services internes.

Le deuxième processus est dédié aux interactions utilisateur. Il contient le service utilisateur, l'orchestrateur multi-agents chargé du traitement des questions médicales, la base des utilisateurs ainsi que la base des conversations. Lorsqu'une requête médicale est reçue, le service utilisateur déclenche l'orchestrateur multi-agents qui interagit avec les services d'intelligence artificielle et la base de connaissances afin de produire une réponse adaptée.

Le troisième processus est consacré à la surveillance des données physiologiques. Il regroupe le service IoT, l'orchestrateur multi-agents d'analyse des mesures et la base des mesures IoT. Les données reçues sont enregistrées immédiatement dans la base de données puis analysées de manière asynchrone afin de détecter d'éventuelles anomalies. Les résultats de l'analyse sont ensuite stockés dans la même base.

Le système s'appuie également sur plusieurs services externes. Le service d'intelligence artificielle fournit le modèle de langage utilisé pour le raisonnement et la génération des réponses. La base de connaissances externe contient les index vectoriels exploités lors des recherches sémantiques. Enfin, le service de messagerie permet l'envoi des emails de réinitialisation du mot de passe lors des procédures de récupération de compte.

Ainsi, ce diagramme met en évidence la répartition physique des composants du système, l'isolation des processus métier ainsi que les échanges entre les ressources internes et les services externes nécessaires au fonctionnement de la plateforme.

2.7 Description des agents

Le système repose sur une architecture multi-agents dans laquelle chaque agent est spécialisé dans une tâche précise du processus de traitement. Afin d'assurer une répartition efficace des responsabilités, les agents interviennent à différentes étapes du traitement des requêtes médicales et de l'analyse des données de santé.

Les agents de santé générale et de santé mentale s'appuient sur une approche de génération augmentée par récupération (*Retrieval-Augmented Generation* ou RAG) afin d'exploiter des ressources documentaires spécialisées et d'enrichir les réponses produites. Les autres agents, à savoir l'agent lecteur et routeur, l'agent pharmacie, l'agent nutrition,

l'agent clinique, l'agent monitoring, l'agent évaluateur, l'agent rédacteur et l'agent manager, utilisent principalement les capacités d'un modèle de langage (LLM) pour accomplir leurs tâches respectives.

Le mécanisme RAG permet d'enrichir les réponses générées par le modèle en s'appuyant sur une base documentaire externe. Lorsqu'une requête est reçue, les informations les plus pertinentes sont d'abord recherchées dans les documents de référence, puis intégrées au contexte fourni au modèle de langage. Cette approche améliore la pertinence des réponses produites et permet de les appuyer sur des ressources documentaires spécialisées. [1]

2.7.1 Agent lecteur et routeur

L'agent lecteur et routeur constitue le point d'entrée du système. Son rôle consiste à analyser les requêtes utilisateur afin d'identifier le domaine médical concerné et d'orienter la demande vers l'agent spécialisé approprié. Cette classification est réalisée à l'aide d'un modèle de langage capable d'interpréter le contenu de la requête et de déterminer le domaine médical correspondant.

Entrée : question soumise par l'utilisateur.

Sortie : domaine médical identifié ainsi que la requête orientée vers l'agent spécialisé correspondant.

2.7.2 Agent de santé générale

L'agent de santé générale fournit une analyse médicale globale venant compléter les réponses spécialisées. Il intervient dans le traitement des questions utilisateur et peut également être sollicité lors de l'analyse d'une anomalie détectée par le système SmartWatch IoT.

Afin d'améliorer la qualité des réponses produites, cet agent combine les capacités d'un modèle de langage avec une approche de génération augmentée par récupération (RAG). Les connaissances exploitées proviennent d'une base documentaire médicale générale construite à partir de l'ouvrage de référence *The Gale Encyclopedia of Medicine* [2], qui couvre un large ensemble de concepts médicaux, de maladies, de symptômes, de traitements et de pratiques de santé. L'exploitation de cette ressource permet à l'agent de fournir des réponses plus pertinentes et mieux contextualisées.

Entrée : question utilisateur ou données médicales associées à une anomalie détectée.

Sortie : analyse médicale générale structurée.

2.7.3 Agent santé mentale

L'agent santé mentale traite les questions liées au stress, à l'anxiété, à la dépression et aux différentes problématiques psychologiques. Pour fournir des réponses adaptées, il s'appuie sur un modèle de langage enrichi par un mécanisme de génération augmentée par récupération (RAG), lui permettant d'exploiter une base documentaire spécialisée en santé mentale.

Les connaissances utilisées proviennent de ressources de référence portant sur les thérapies cognitivo-comportementales, la gestion de l'anxiété, la dépression et le bien-être psychologique. La base documentaire exploitée par l'agent est constituée du *Cognitive Behavioural Therapy (CBT) Skills Workbook* [3], de l'ouvrage *The Cognitive Behavioral*

Workbook for Anxiety de William J. Knaus [4], ainsi que du guide *Depression* publié par le National Institute of Mental Health (NIMH) [5]. Ces ressources permettent à l'agent de s'appuyer sur des connaissances reconnues afin de fournir des réponses plus fiables, contextualisées et cohérentes.

Entrée : question relevant du domaine de la santé mentale.

Sortie : réponse spécialisée accompagnée de recommandations adaptées.

2.7.4 Agent pharmacie

L'agent pharmacie est spécialisé dans les questions relatives aux médicaments, aux effets secondaires et aux interactions médicamenteuses. Il utilise un modèle de langage afin d'analyser la demande et de générer une réponse adaptée au contexte présenté par l'utilisateur.

Son objectif est de fournir des informations générales permettant d'éclairer l'utilisateur sans se substituer à l'avis d'un professionnel de santé.

Entrée : question relevant du domaine pharmaceutique.

Sortie : réponse pharmaceutique adaptée à la demande de l'utilisateur.

2.7.5 Agent nutrition

L'agent nutrition intervient pour les questions liées à l'alimentation, aux habitudes nutritionnelles et à l'hygiène de vie. Il utilise un modèle de langage pour formuler des recommandations adaptées au contexte présenté par l'utilisateur et contribuer à la promotion de bonnes pratiques alimentaires.

Entrée : question relevant du domaine nutritionnel.

Sortie : recommandations nutritionnelles et conseils alimentaires.

2.7.6 Agent clinique

L'agent clinique est chargé d'analyser les questions relatives aux symptômes et aux situations médicales générales. Il intervient également lors du traitement des anomalies détectées par le système SmartWatch IoT afin d'apporter une interprétation médicale prudente des données observées.

Cet agent s'appuie sur un modèle de langage pour produire une analyse informative tout en évitant l'établissement de diagnostics médicaux définitifs ou la prescription de traitements.

Entrée : question clinique ou données médicales associées à une anomalie détectée.

Sortie : analyse clinique et recommandations générales.

2.7.7 Agent monitoring

L'agent monitoring constitue le composant central du scénario SmartWatch IoT et représente le premier niveau d'analyse des données physiologiques reçues. Il reçoit les mesures collectées par la montre connectée et examine leur évolution afin d'identifier d'éventuelles anomalies ou situations nécessitant une attention particulière.

Pour réaliser cette tâche, l'agent s'appuie sur un modèle de langage chargé d'interpréter les données reçues et de les confronter aux règles de surveillance définies par le système. Son rôle consiste à vérifier la cohérence des mesures, à détecter les anomalies potentielles et à produire une première évaluation de l'état de santé de l'utilisateur. Cette analyse

repose directement sur les données physiologiques reçues ainsi que sur l'historique des mesures enregistrées.

Entrée : données physiologiques collectées par la montre connectée ainsi que l'historique des mesures.

Sortie : statut de surveillance, anomalies détectées et observations associées.

2.7.8 Agent évaluateur

L'agent évaluateur intervient après la génération des réponses ou des analyses. Sa mission consiste à vérifier leur qualité, leur cohérence et leur pertinence avant leur transmission à l'utilisateur.

Pour réaliser cette validation, il s'appuie sur un modèle de langage capable d'évaluer les résultats produits par les autres agents et de déterminer si ceux-ci satisfont les critères de qualité définis.

Entrée : réponses ou analyses produites par les agents spécialisés.

Sortie : validation ou demande de correction.

2.7.9 Agent rédacteur

L'agent rédacteur est responsable de la génération du résultat final destiné à l'utilisateur. Il intervient après la phase de validation afin de produire une réponse claire, cohérente et facilement compréhensible.

Pour accomplir cette tâche, il utilise un modèle de langage chargé de fusionner les différentes analyses validées et de reformuler les informations sous une forme adaptée à l'utilisateur. Selon le contexte, il peut générer soit une réponse médicale détaillée dans le scénario de réponse utilisateur, soit une alerte concise dans le scénario SmartWatch IoT.

Entrée : réponses validées par l'agent évaluateur.

Sortie : réponse finale ou alerte destinée à l'utilisateur.

2.7.10 Agent manager

L'agent manager intervient lorsqu'une réponse ou une analyse est jugée insuffisante par l'agent évaluateur. Il est chargé d'identifier l'origine du problème, de relancer certains traitements ou de demander une correction avant une nouvelle phase de validation.

Grâce à l'utilisation d'un modèle de langage, il contribue à améliorer la robustesse du système et à garantir la qualité des résultats transmis à l'utilisateur.

Entrée : réponse ou analyse rejetée par l'agent évaluateur.

Sortie : version corrigée ou améliorée du résultat.

Conclusion

Ce chapitre a permis de présenter l'analyse et la conception du système à travers différents diagrammes UML décrivant ses fonctionnalités, sa structure de données, son architecture logicielle et son déploiement. Il a également permis de détailler le fonctionnement des principaux agents intelligents intervenant dans le traitement des requêtes médicales et l'analyse des données physiologiques. L'ensemble de ces modèles constitue une base essentielle pour la réalisation et l'implémentation de la solution, qui seront présentées dans le chapitre suivant.

Chapitre 3

Réalisation et Résultats

Introduction

Ce chapitre présente la réalisation de la plateforme de surveillance médicale intelligente développée dans le cadre de ce projet. Il décrit l'environnement matériel et logiciel utilisé, les principales technologies intégrées ainsi que l'architecture d'intelligence artificielle mise en œuvre. Les différentes interfaces développées sont ensuite présentées afin d'illustrer les fonctionnalités offertes aux utilisateurs. Enfin, une phase de validation est menée pour évaluer les performances du système, notamment la pertinence des réponses médicales, le fonctionnement de l'architecture multi-agents, la surveillance des données IoT et l'apport du mécanisme RAG.

3.1 Environnement matériel

Le développement, les tests et l'exécution de l'application ont été réalisés sur la configuration matérielle présentée dans le tableau suivant.

Table 3.1 – Configuration matérielle utilisée

Composant	**Caractéristiques**
Processeur	Intel® Core™ i7-10610U @ 1.80 GHz (jusqu'à 2.30 GHz)
Mémoire RAM	16 Go DDR4 (3200 MHz)
Stockage	SSD 238 Go
Carte graphique	Intel® UHD Graphics (128 Mo)
Système d'exploitation	Windows 11 Pro 64 bits

Cette configuration a permis d'assurer le développement et l'exécution simultanée des différents services constituant l'application. Elle a également fourni les ressources nécessaires au traitement des données médicales, à l'exécution des agents intelligents ainsi qu'à la gestion des communications entre les différentes composantes du système.

3.2 Environnement logiciel

Le développement de la plateforme de surveillance médicale intelligente a nécessité l'utilisation de plusieurs technologies logicielles couvrant les différentes couches du système. Ces technologies interviennent dans le développement de l'interface utilisateur, la gestion des services backend, l'orchestration des agents intelligents, la recherche sémantique ainsi que le stockage des données.

3.2.1 Python

Python est un langage de programmation de haut niveau reconnu pour sa simplicité, sa lisibilité et sa richesse en bibliothèques. Grâce à son vaste écosystème, il est largement utilisé dans les domaines du développement web, de l'intelligence artificielle, de l'analyse de données et de l'Internet des Objets (IoT) [6].

Dans le cadre de ce projet, Python constitue le langage principal utilisé pour le développement du backend et l'implémentation des différentes fonctionnalités intelligentes du système. Il a permis de développer les services web avec FastAPI, de mettre en œuvre l'architecture multi-agents à l'aide de CrewAI, d'intégrer le mécanisme RAG avec LangChain et Pinecone, ainsi que de gérer le traitement des données médicales issues des dispositifs connectés.

Le choix de Python repose principalement sur sa simplicité de développement et son vaste écosystème de bibliothèques dédiées à l'intelligence artificielle. Comparativement à des langages tels que Java ou C++, Python permet un développement plus rapide tout en offrant une excellente intégration avec les frameworks utilisés dans ce projet. Cette caractéristique a facilité l'implémentation des agents intelligents, des API web et des mécanismes d'analyse des données médicales.



Figure 3.1 – Logo Python

3.2.2 React.js

React.js est une bibliothèque JavaScript open source destinée au développement d'interfaces utilisateur interactives et dynamiques. Basée sur une architecture orientée composants, elle facilite la création d'applications web modernes tout en améliorant la maintenabilité du code.[7]

Dans ce projet, React.js a été utilisé pour développer l'ensemble de l'interface utilisateur, notamment les pages d'authentification, l'assistant médical intelligent, le tableau de bord de surveillance des constantes vitales ainsi que le système d'affichage des alertes médicales.

React.js a été retenu pour son architecture basée sur les composants réutilisables, qui facilite le développement, la maintenance et l'évolution de l'application. Il permet également de construire des interfaces utilisateur interactives et dynamiques tout en améliorant l'organisation du code. De plus, sa large communauté et sa documentation abondante ont constitué un avantage important lors du développement du frontend.

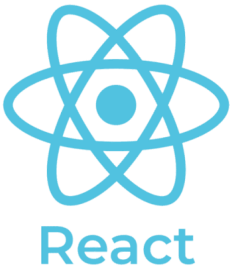


FIGURE 3.2 – Logo React.js

3.2.3 FastAPI

FastAPI est un framework moderne de développement d'API REST basé sur le langage Python. Il offre de hautes performances tout en permettant la création rapide de services web sécurisés et robustes.[8]

Dans notre système, FastAPI constitue le noyau du backend. Il assure la gestion des utilisateurs, l'authentification, le traitement des requêtes, la réception des données provenant des dispositifs connectés ainsi que la communication avec les différents agents intelligents.

Le choix de FastAPI s'explique par ses performances élevées et sa simplicité d'utilisation. Comparativement à Flask, il fournit nativement la validation des données, la génération automatique de la documentation des API ainsi que le support de la programmation asynchrone. Par rapport à Django, il est plus léger et particulièrement adapté au développement d'API REST modernes. Ces caractéristiques en font une solution pertinente pour une architecture basée sur des services web et des agents intelligents. De plus, le support natif des opérations asynchrones permet de traiter efficacement plusieurs requêtes simultanées, ce qui est particulièrement utile dans une architecture composée de plusieurs services et agents collaboratifs.



FIGURE 3.3 – Logo FastAPI

3.2.4 CrewAI

CrewAI est un framework spécialisé dans l'orchestration des systèmes multi-agents basés sur les modèles de langage. Il permet de coordonner plusieurs agents collaboratifs afin de résoudre une tâche complexe [9].

Dans le cadre de ce projet, CrewAI est utilisé pour implémenter l'architecture multi-agents médicale. Chaque agent dispose d'une responsabilité spécifique, notamment l'analyse des requêtes, la vérification des informations, l'évaluation des résultats ainsi que la génération de la réponse finale.

Le choix de CrewAI a été motivé par sa simplicité de conception des systèmes multi-agents et sa facilité d'intégration avec les modèles de langage modernes. Plusieurs solutions ont été étudiées, notamment CrewAI et LangGraph. Bien que LangGraph offre une grande flexibilité grâce à une architecture basée sur des graphes d'états permettant de construire des flux complexes, sa mise en œuvre nécessite une configuration plus avancée et une gestion plus détaillée des transitions entre agents.

À l'inverse, CrewAI adopte une approche orientée rôles où chaque agent possède un objectif, une mission et des responsabilités clairement définies. Cette approche correspond parfaitement aux besoins du système développé, qui repose sur plusieurs agents médicaux spécialisés collaborant pour analyser une requête utilisateur ou des données physiologiques. Grâce à sa simplicité, sa lisibilité et sa rapidité de développement, CrewAI a été retenu comme solution d'orchestration des agents intelligents au sein de la plateforme.



Figure 3.4 – Logo CrewAI

3.2.5 LangChain

LangChain est un framework conçu pour faciliter le développement d'applications exploitant les modèles de langage de grande taille (LLM). Il fournit plusieurs outils permettant l'intégration de sources de connaissances externes et l'orchestration des flux de traitement.[10]

Dans cette application, LangChain est utilisé pour gérer les interactions avec les modèles d'intelligence artificielle et mettre en œuvre le mécanisme de recherche augmentée par récupération (RAG).

LangChain a été choisi pour son intégration native avec les modèles OpenAI, les bases vectorielles et les systèmes RAG. Il fournit un ensemble complet d'outils permettant de gérer efficacement les embeddings, les recherches sémantiques et la construction de chaînes de traitement complexes, ce qui a considérablement simplifié l'implémentation du système.



Figure 3.5 – Logo LangChain

3.2.6 OpenAI GPT-4o Mini

GPT-4o Mini est un modèle de langage développé par OpenAI capable de comprendre et de générer du texte en langage naturel avec un niveau élevé de précision et de cohérence [11].

Dans notre système, ce modèle constitue le moteur principal d'intelligence artificielle. Il est sollicité pour analyser les questions médicales des utilisateurs, interpréter les données contextuelles et produire des réponses adaptées aux besoins exprimés.

Le choix de GPT-4o Mini repose sur un compromis entre qualité des réponses, rapidité d'exécution et coût d'utilisation. Comparativement aux modèles plus volumineux tels que GPT-4o ou GPT-4.1, GPT-4o Mini présente un coût nettement plus faible tout en conservant un niveau de performance élevé pour les tâches de compréhension et de génération de texte.

Cette caractéristique est particulièrement importante dans une architecture multi-agents où plusieurs appels au modèle peuvent être réalisés pour une seule requête utilisateur. De plus, son temps de réponse réduit améliore considérablement l'expérience utilisateur.

Concernant la confidentialité, seules les informations strictement nécessaires au traitement des requêtes sont transmises au modèle. Les données sensibles telles que les comptes utilisateurs, les historiques de conversations et les mesures IoT demeurent stockées localement dans la base SQLite du système. Seules les informations nécessaires à la génération de la réponse sont transmises au modèle de langage.



Figure 3.6 – Logo OpenAI GPT-4o Mini

3.2.7 Pinecone

Pinecone est une base de données vectorielle optimisée pour le stockage et la recherche rapide d'embeddings. Elle facilite la recherche sémantique au sein d'importants volumes de données textuelles.[12]

Dans ce projet, Pinecone est utilisée pour stocker les représentations vectorielles des documents médicaux. Lorsqu'une question est posée, elle permet d'identifier rapidement les informations les plus pertinentes afin d'enrichir le contexte transmis au modèle de langage.

Le choix de Pinecone s'explique par ses performances élevées en recherche vectorielle ainsi que par sa compatibilité native avec LangChain. Comparativement à certaines solutions auto-hébergées telles que ChromaDB ou Weaviate, Pinecone simplifie la gestion de l'infrastructure tout en garantissant des temps de réponse très faibles, même lorsque le volume des documents augmente.



FIGURE 3.7 – Logo Pinecone

3.2.8 SQLite

SQLite est un système de gestion de base de données relationnelle léger intégré directement dans l'application.[13]

Il est utilisé pour stocker les comptes utilisateurs, l'historique des conversations, les mesures médicales collectées par les objets connectés ainsi que les résultats d'analyse produits par les agents intelligents.

SQLite a été retenu pour sa simplicité de déploiement et sa faible consommation de ressources. Contrairement à MySQL ou PostgreSQL, il ne nécessite pas de serveur dédié et fonctionne directement à partir d'un fichier local. Dans le contexte de ce projet académique, le volume de données reste modéré, ce qui rend SQLite parfaitement adapté tout en simplifiant l'installation, la maintenance et le déploiement de l'application.



FIGURE 3.8 – Logo SQLite

3.3 Architecture de l'intelligence artificielle

3.3.1 Création des agents intelligents

Le système repose sur neuf agents intelligents distincts, chacun spécialisé dans un rôle précis au sein du pipeline de traitement. Chaque agent est instancié de manière indépendante à chaque requête, ce qui garantit l'isolation totale entre le pipeline utilisateur et le pipeline de surveillance IoT, évitant ainsi toute contamination de l'historique entre les deux flux de traitement.

Configuration des agents

Tous les agents s'appuient sur le même modèle de langage GPT-4o Mini, configuré avec un paramètre de température adapté à la nature de chaque tâche. Une température faible favorise des réponses déterministes et structurées (agents de routage et d'évaluation),

tandis qu'une température plus élevée permet des réponses plus nuancées et empathiques (agents de rédaction et de soutien). Le tableau 3.2 présente la configuration de chaque agent.

Table 3.2 – Configuration des agents du système

Agent	Rôle	Temp.	Accès RAG
Agent routeur	Classifier la requête en domaine	0.0	Aucun
Agent clinique	Médecine clinique et symptômes	0.3	Aucun
Agent santé mentale	Psychologie et soutien émotionnel	0.4	Index santé mentale
Agent nutrition	Diététique et alimentation	0.4	Aucun
Agent pharmacie	Médicaments et interactions	0.3	Aucun
Agent santé générale	Vue clinique globale	0.3	Index médecine générale
Agent évaluateur	Contrôle qualité des réponses	0.1	Aucun
Agent rédacteur	Rédaction de la réponse finale	0.5	Aucun
Agent manager	Correction en cas de réponse insuffisante	0.2	Aucun

Structure d'un agent

Chaque agent est défini par trois attributs fondamentaux imposés par le framework CrewAI :

- **Rôle** : l'identité et la spécialité de l'agent, qui oriente son comportement global ;
- **Objectif** : la mission précise que l'agent cherche à accomplir dans le système ;
- **Contexte** : la description de son expertise et de ses contraintes, qui influence le style et la profondeur de ses réponses.

À titre d'exemple, l'agent routeur opère avec une température de 0,0 — la plus faible du système — car sa tâche se limite à classifier la requête dans l'un des cinq domaines médicaux, sans aucune marge d'interprétation. À l'opposé, l'agent rédacteur utilise une température de 0,5 afin de produire des réponses naturelles, chaleureuses et adaptées au profil de l'utilisateur.

Format d'échange entre agents

Les agents communiquent par transmission de contexte : chaque agent reçoit automatiquement les sorties des agents qui le précèdent dans le pipeline. Le format de sortie varie selon le rôle de l'agent : les agents de routage et d'évaluation produisent un objet JSON structuré permettant un traitement automatique, tandis que les agents spécialistes, rédacteur et manager produisent du texte libre destiné à être présenté directement à l'utilisateur. Le tableau 3.3 présente le format de sortie attendu de chaque agent.

Table 3.3 – Format de sortie de chaque agent

Agent	Format de sortie
Agent routeur	Objet JSON contenant le domaine détecté (*clinical, nutrition, pharmacy, mental_health ou general*) ainsi que la justification du choix.
Agent spécialiste	Texte structuré contenant une réponse médicale détaillée dans le domaine concerné.
Agent santé générale	Texte structuré fournissant une analyse médicale générale complémentaire.
Agent évaluateur	Objet JSON contenant le statut de validation, le score attribué à la réponse ainsi que les éventuelles recommandations d'amélioration.
Agent rédacteur	Texte final reformulé, clair et empathique, destiné directement à l'utilisateur.
Agent manager	Texte corrigé ou enrichi lorsque la réponse initiale est jugée insuffisante par l'agent évaluateur.

3.3.2 Base de connaissances vectorielle

La base de connaissances du système repose sur la plateforme Pinecone, utilisée pour le stockage et l'indexation des représentations vectorielles des documents médicaux exploités par le mécanisme RAG. Afin d'améliorer la pertinence des informations récupérées, deux index distincts ont été créés, chacun étant dédié à un domaine médical spécifique.

Processus de vectorisation

Avant leur stockage dans Pinecone, les documents médicaux sont soumis à une phase de prétraitement. Les documents sont d'abord découpés en segments de taille réduite appelés *chunks* afin de faciliter la recherche sémantique et de limiter la quantité d'information transmise au modèle de langage lors de la génération des réponses.

Chaque segment est ensuite converti en représentation vectorielle à l'aide du modèle d'embedding `all-MiniLM-L6-v2`, qui produit des vecteurs de dimension 384. Les vecteurs générés sont alors stockés dans Pinecone avec les métadonnées associées au document source.

Lorsqu'une question est soumise au système, celle-ci est également transformée en vecteur puis comparée aux vecteurs présents dans l'index à l'aide de la similarité cosinus. Les segments les plus pertinents sont récupérés et transmis aux agents spécialisés afin d'enrichir le contexte utilisé pour la génération de la réponse.

Le découpage des documents est réalisé avec une taille de 500 caractères et un chevauchement de 50 caractères entre deux segments successifs. Ce chevauchement permet de conserver le contexte entre les différentes portions d'un même document et de réduire la perte d'information lors du découpage.

Index `mental-health`

Cet index regroupe les ressources spécialisées en santé mentale utilisées par le *Mental Health Agent*. Les principales sources documentaires intégrées sont :

- **CBT Skills Workbook** ;
- **Anxiety CBT Workbook** ;
- **NIMH – Depression** ;
- **Mental Health Guide**.

Ces documents couvrent principalement les troubles anxieux, la dépression ainsi que les approches thérapeutiques basées sur la thérapie cognitivo-comportementale.

Index `medical-chatbot`

Cet index contient les ressources de médecine générale utilisées par le *General Health Agent*. Il est principalement alimenté par un ouvrage médical de référence couvrant un large éventail de pathologies, de symptômes, de diagnostics et de traitements généraux.

Le tableau 3.4 présente les principales caractéristiques des deux index vectoriels utilisés dans le système.

Table 3.4 – Caractéristiques des index Pinecone

Paramètre	**mental-health**	**medical-chatbot**
Modèle d'embedding	`all-MiniLM-L6-v2` (384 dimensions)	
Métrique de similarité	Cosinus	
Infrastructure	Serverless – AWS us-east-1	
Taille des chunks	500 caractères	
Chevauchement	50 caractères	
Agent associé	Mental Health Agent	General Health Agent
Sources documentaires	Ressources CBT, NIMH et guides de santé mentale	Ouvrage de médecine générale

La figure suivante présente les index vectoriels créés dans Pinecone pour le stockage de la base de connaissances du système.

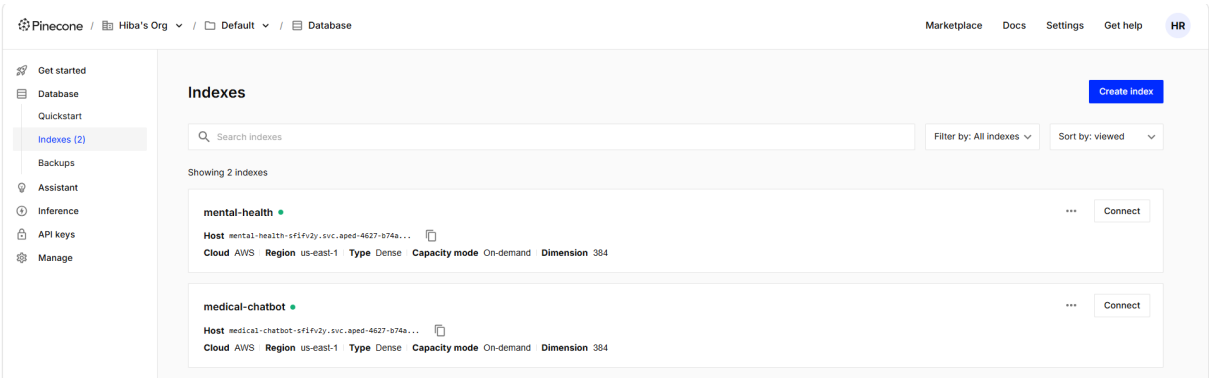


Figure 3.9 – Vue d'ensemble des index vectoriels dans la console Pinecone

La séparation entre les deux index permet d'améliorer la précision des informations récupérées et facilite le routage des requêtes vers l'agent spécialisé approprié. Lorsqu'une question est classée dans le domaine de la santé mentale, l'index `mental-health` est interrogé afin de récupérer les informations les plus pertinentes. En parallèle, l'index `medical-chatbot` fournit une base de connaissances générale utilisée pour compléter l'analyse médicale globale réalisée par le système.

3.4 Présentation de l'interface de l'application

L'application développée propose plusieurs interfaces permettant à l'utilisateur d'interagir facilement avec le système. Ces interfaces ont été conçues afin d'assurer une expérience utilisateur intuitive et un accès rapide aux différentes fonctionnalités de la plateforme.

3.4.1 Interface de connexion

L'interface de connexion permet aux utilisateurs déjà enregistrés d'accéder à leur espace personnel. Pour s'authentifier, l'utilisateur doit saisir son adresse électronique ainsi que son mot de passe. Une fois les informations validées, l'accès aux différentes fonctionnalités de la plateforme est autorisé.

Cette interface constitue le point d'entrée principal de l'application et garantit un accès sécurisé aux données médicales et aux services proposés.

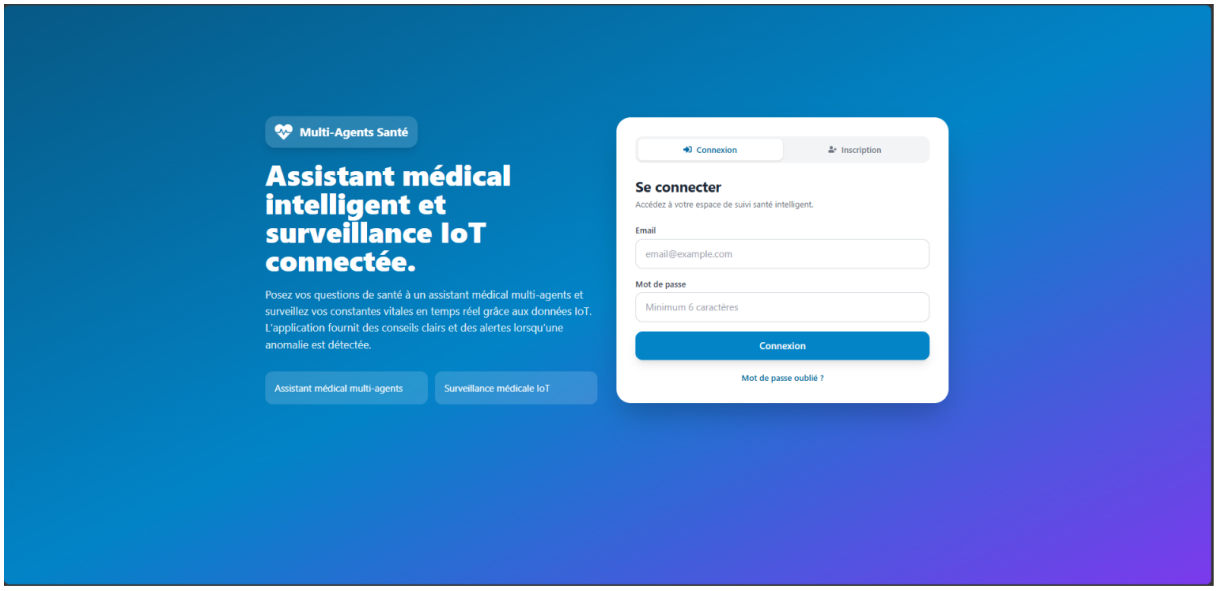


FIGURE 3.10 – Interface de connexion

3.4.2 Interface d'inscription

L'interface d'inscription permet aux nouveaux utilisateurs de créer un compte afin d'accéder aux fonctionnalités de la plateforme. Lors de cette étape, l'utilisateur renseigne les informations nécessaires telles que son nom complet, son adresse électronique et son mot de passe.

Après validation du formulaire, un nouveau compte est créé et l'utilisateur peut ensuite se connecter à son espace personnel.

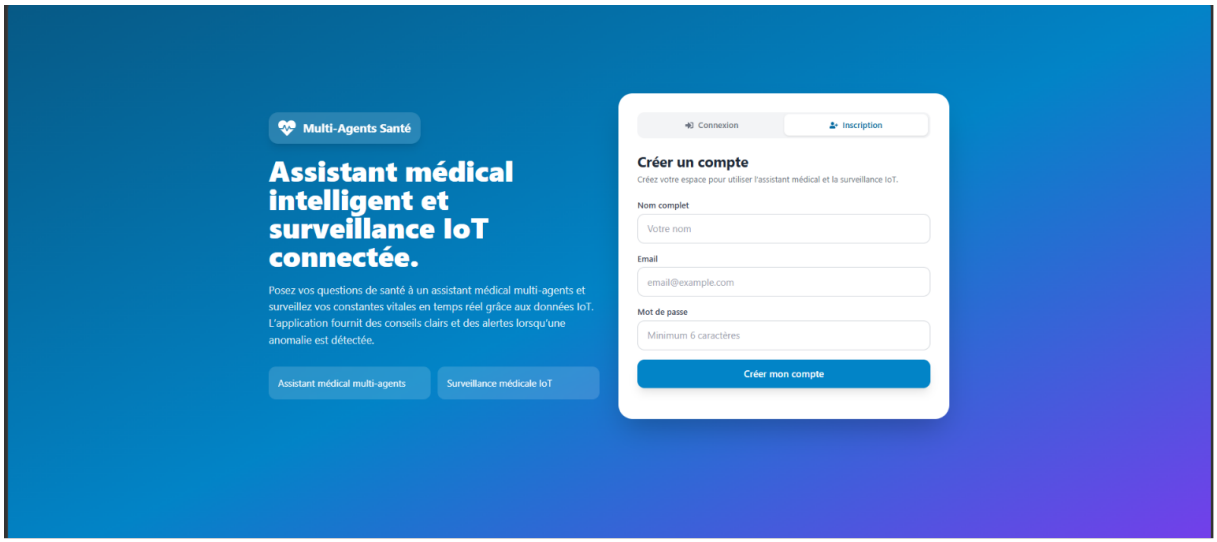


FIGURE 3.11 – Interface d'inscription

3.4.3 Interface de récupération du mot de passe

Afin d'améliorer la sécurité et l'accessibilité du système, une fonctionnalité de récupération du mot de passe a été mise en place. L'utilisateur peut demander un lien de réinitialisation envoyé par courrier électronique.

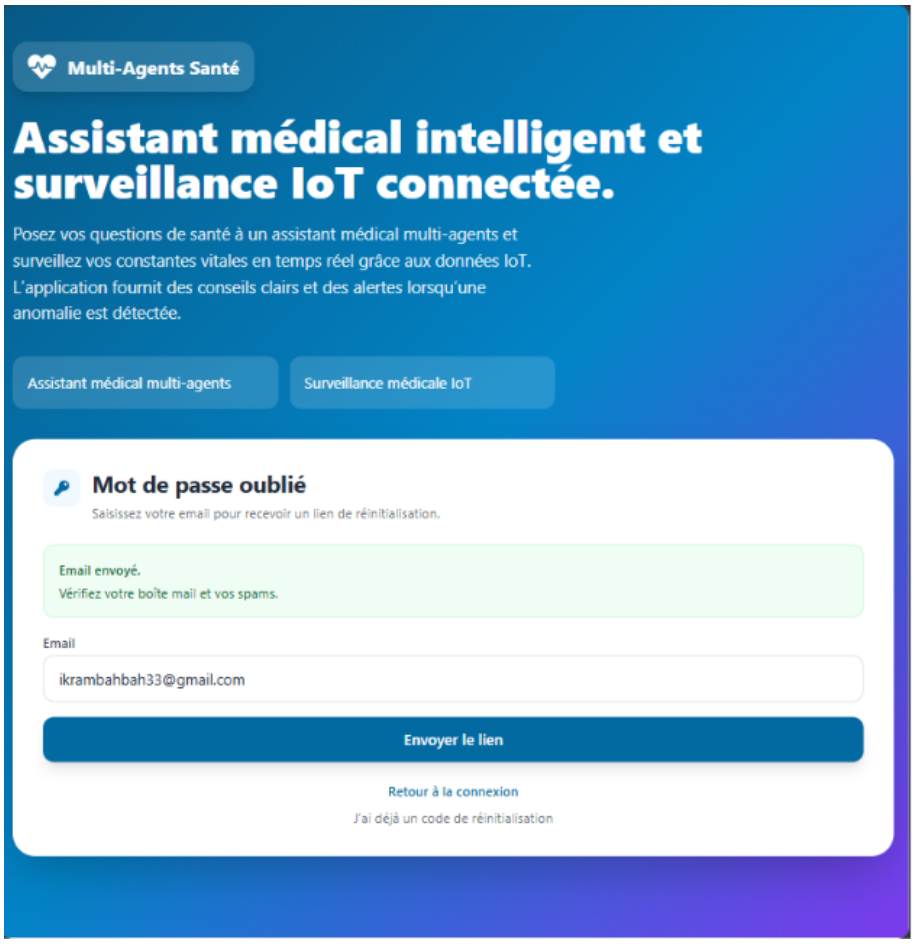


FIGURE 3.12 – Interface de récupération du mot de passe

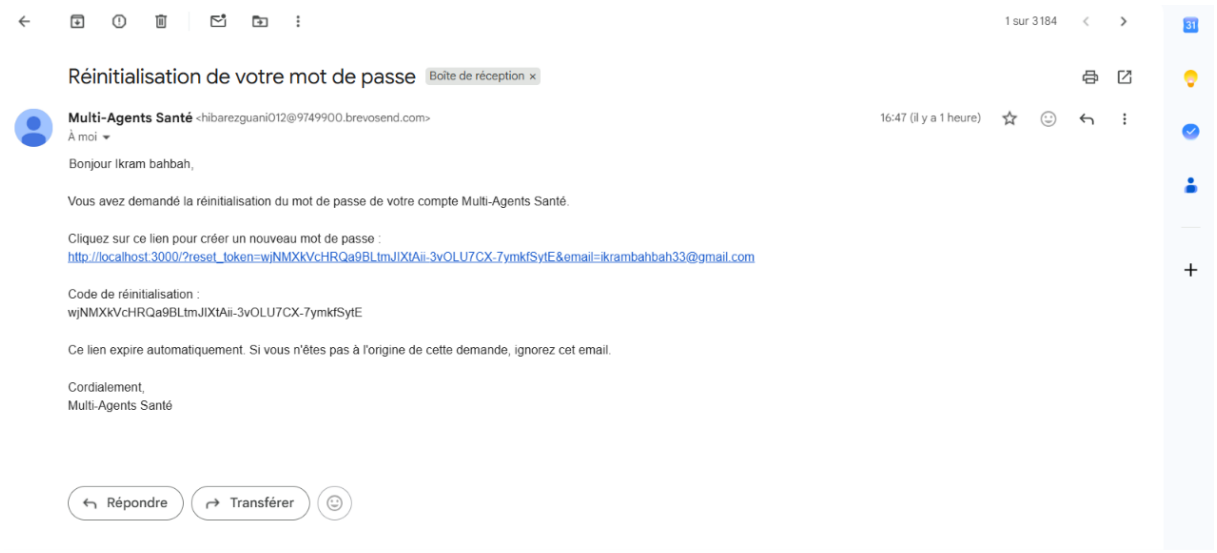


FIGURE 3.13 – Courriel de réinitialisation du mot de passe



FIGURE 3.14 – Interface de réinitialisation du mot de passe

3.4.4 Interface d'accueil

Après authentification, l'utilisateur accède à la page d'accueil de l'application. Cette interface permet d'accéder rapidement aux principales fonctionnalités offertes par le système.

- Assistant médical intelligent ;
- Surveillance des données IoT ;
- Gestion des alertes médicales.

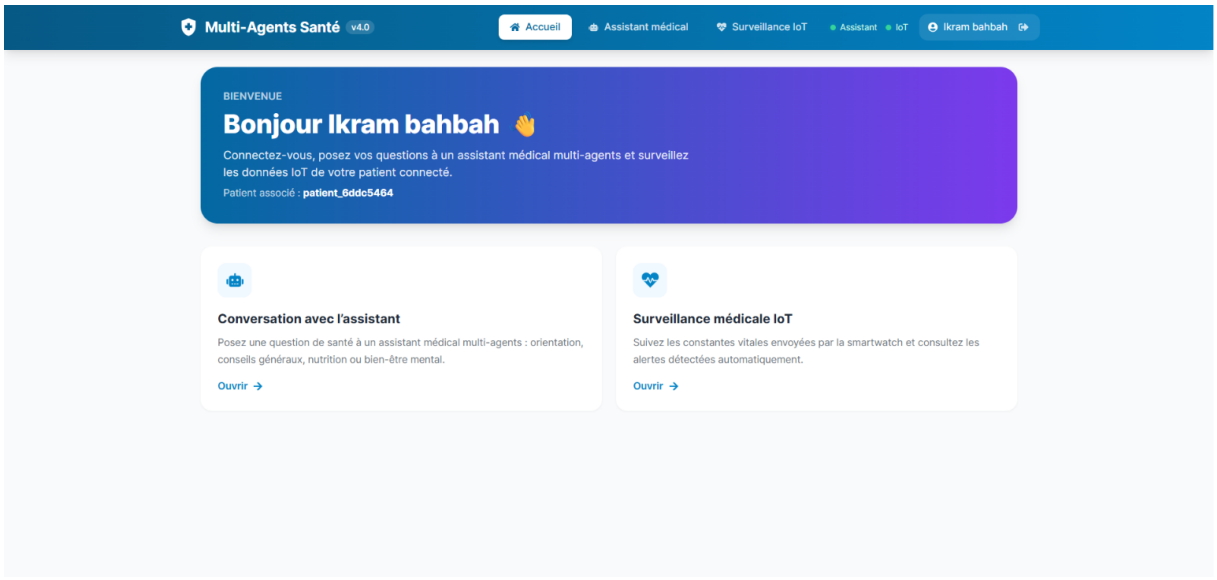


FIGURE 3.15 – Interface d'accueil

3.4.5 Assistant médical intelligent

Cette interface permet à l'utilisateur de communiquer avec l'assistant médical basé sur l'architecture multi-agents. Les questions soumises sont analysées puis traitées par les différents agents afin de fournir une réponse pertinente et contextualisée.

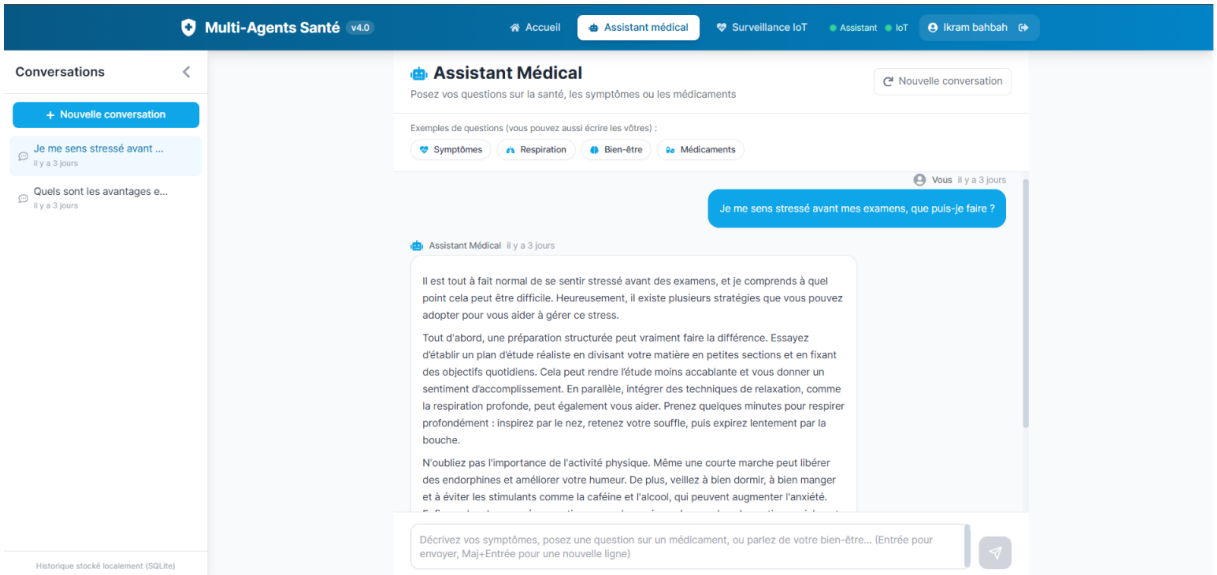


FIGURE 3.16 – Interface de l'assistant médical intelligent

3.4.6 Interface de surveillance médicale IoT

Cette interface assure le suivi en temps réel des constantes vitales provenant de la montre connectée. Les informations affichées comprennent la fréquence cardiaque, la saturation en oxygène, la pression artérielle et la température corporelle.

Les données sont représentées sous forme de graphiques afin de faciliter leur interprétation et de permettre une surveillance continue de l'état de santé du patient.

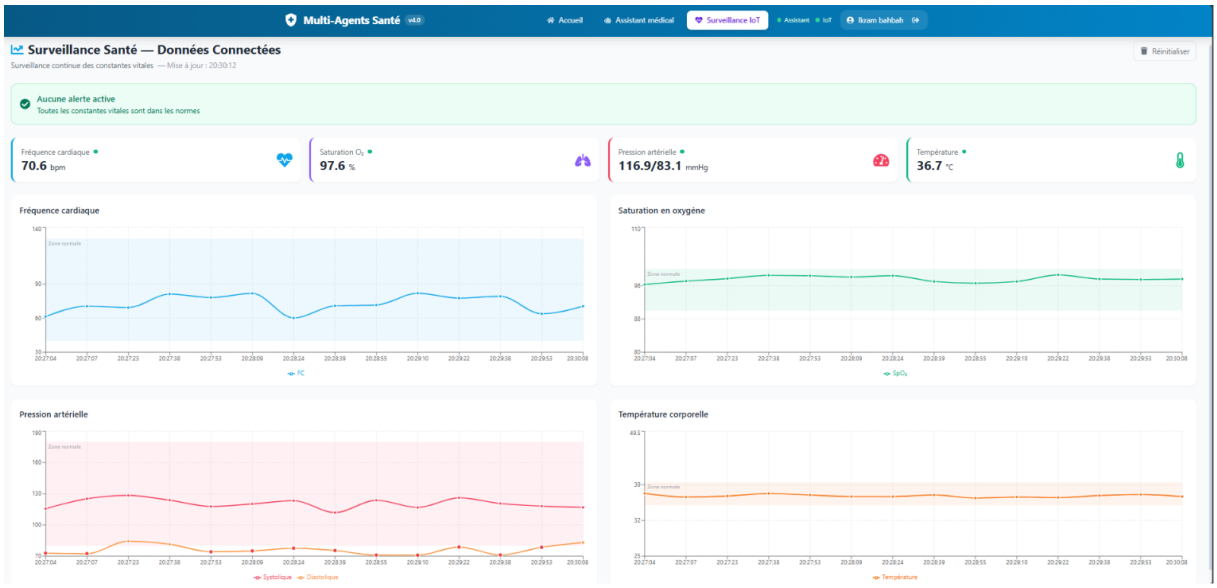


FIGURE 3.17 – Interface de surveillance des données IoT

3.4.7 Système d'alerte médicale

Lorsqu'une mesure dépasse les seuils médicaux prédéfinis, le système détecte automatiquement une anomalie et génère une alerte.

L'alerte précise le patient concerné, la valeur anormale détectée ainsi qu'une recommandation préliminaire destinée à attirer l'attention sur la situation observée.

Dans l'exemple présenté, une fréquence cardiaque de 150 bpm a été détectée, ce qui dépasse la plage normale définie par le système et déclenche automatiquement une alerte médicale.

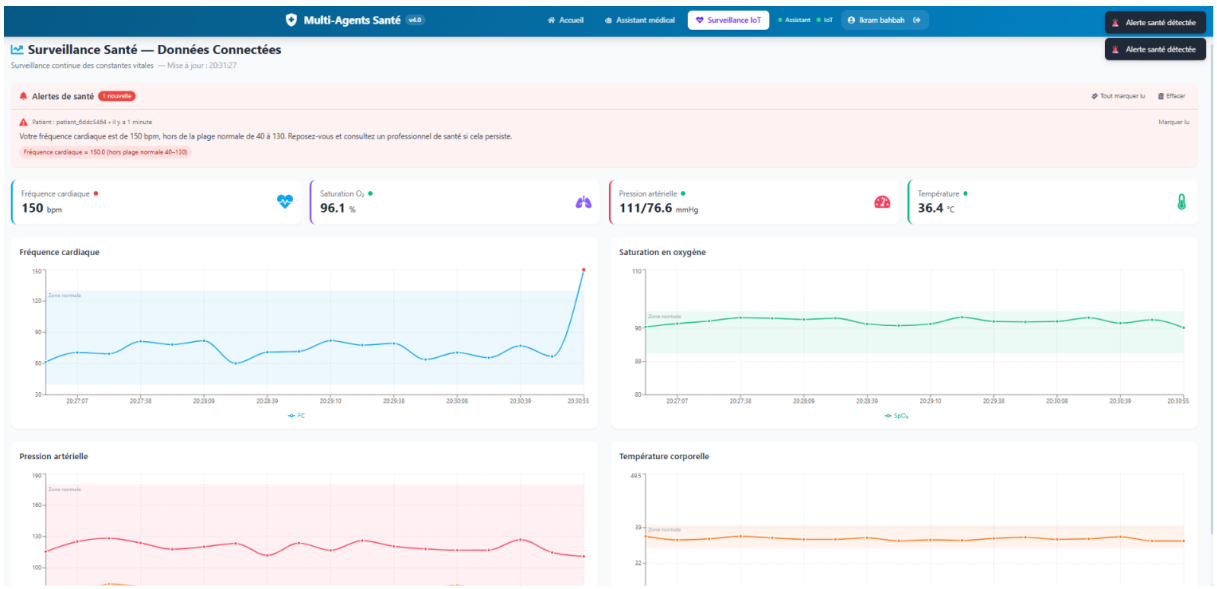


FIGURE 3.18 – Exemple d'alerte médicale générée automatiquement

3.5 Validation du système

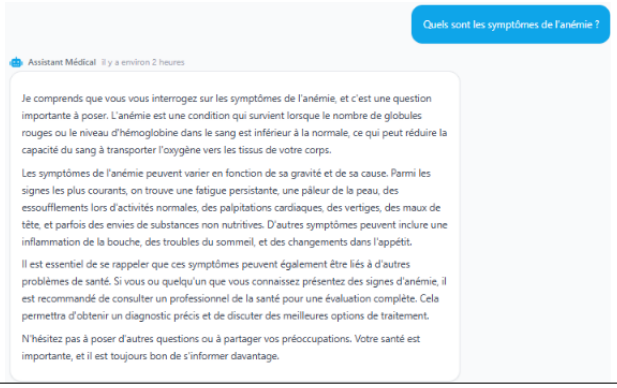
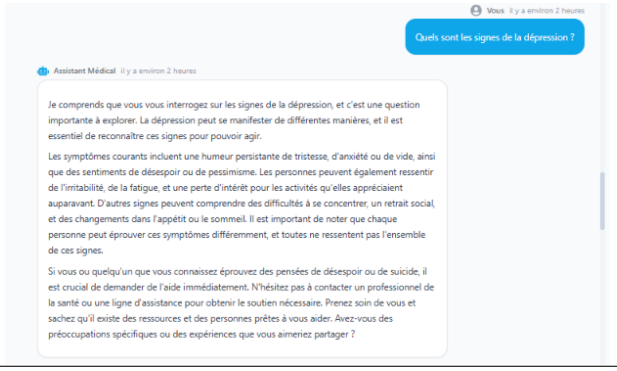
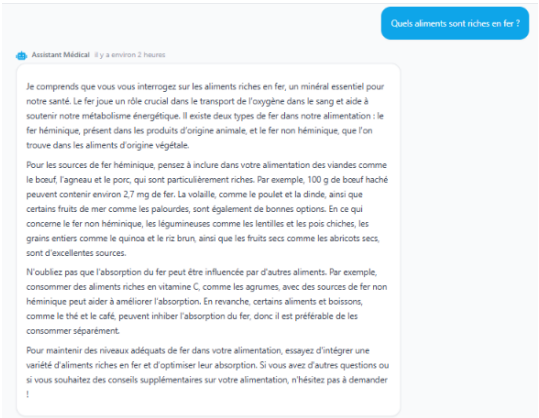
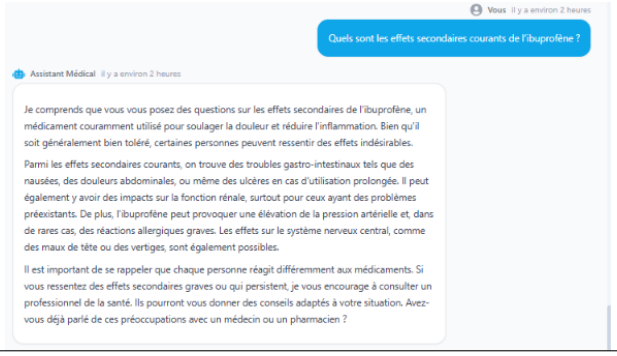
Afin d'évaluer les performances du système développé, une phase de validation a été réalisée.Cette étape a pour objectif de vérifier la pertinence des réponses médicales générées par le système, de valider le fonctionnement de l'architecture multi-agents, d'évaluer les capacités de surveillance IoT ainsi que de mesurer l'apport du mécanisme RAG dans l'amélioration des réponses produites.

3.5.1 Validation de la pertinence des réponses médicales

Pour ce faire, plusieurs questions couvrant différents domaines médicaux ont été soumises au système, notamment la santé générale, la santé mentale, la nutrition et la pharmacie. Les réponses obtenues ont ensuite été analysées afin de vérifier leur cohérence et leur pertinence.

Le tableau 3.5 présente quelques exemples de questions utilisées lors de la phase de validation, accompagnées des réponses générées par le système et de leur évaluation.

Table 3.5 – Validation de la pertinence des réponses médicales

Question testée	Réponse générée	Validation
Quels sont les symptômes de l'anémie ?		Les principaux symptômes de l'anémie ont été correctement identifiés, notamment la fatigue, la pâleur, les vertiges et l'essoufflement. La réponse est jugée pertinente et cohérente.
Quels sont les signes de la dépression ?		La réponse décrit correctement les symptômes majeurs de la dépression et recommande une consultation professionnelle en cas de besoin.
Quels aliments sont riches en fer ?		Les principales sources alimentaires de fer sont correctement citées. Les recommandations fournies sont adaptées à la question posée.
Quels sont les effets secondaires courants de l'ibuprofène ?		Les effets secondaires les plus fréquents ainsi que certains risques associés à une utilisation prolongée sont correctement présentés.

L'analyse des résultats obtenus montre que le système fournit des réponses cohérentes

et adaptées aux différentes questions soumises. Les informations générées sont globalement conformes aux connaissances médicales exploitées par les agents spécialisés.

Par ailleurs, les réponses produites adoptent une formulation prudente en rappelant que les informations fournies ne remplacent pas l'avis d'un professionnel de santé. Cette approche contribue à améliorer la qualité globale du système tout en limitant les risques liés à une mauvaise interprétation des informations médicales.

Les résultats de cette phase de validation confirment la capacité du système à fournir une assistance médicale pertinente dans plusieurs domaines de la santé. Les réponses générées sont cohérentes avec les connaissances médicales attendues et démontrent l'efficacité de l'architecture multi-agents mise en œuvre.

3.5.2 Validation du chemin de traitement des requêtes

Cette validation vise à vérifier que chaque question suit correctement le chemin prévu dans l'architecture multi-agents. Après l'analyse de la requête par l'agent lecteur et routeur, la question est transmise à l'agent spécialisé correspondant au domaine détecté. En parallèle, l'agent de santé générale est également sollicité afin de fournir une analyse complémentaire. Les réponses sont ensuite évaluées puis fusionnées par l'agent rédacteur avant d'être retournées à l'utilisateur.

Table 3.6 – Validation du chemin de traitement des requêtes

Question testée	**Chemin attendu**	**Résultat**
Quels sont les symptômes de l'anémie ?	Agent lecteur et routeur → Agent clinique + Agent santé générale → Agent évaluateur → Agent rédacteur	Correct
Quels aliments sont riches en fer ?	Agent lecteur et routeur → Agent nutrition + Agent santé générale → Agent évaluateur → Agent rédacteur	Correct
Je souffre d'anxiété depuis plusieurs semaines.	Agent lecteur et routeur → Agent santé mentale + Agent santé générale → Agent évaluateur → Agent rédacteur	Correct
Quels sont les effets secondaires du paracétamol ?	Agent lecteur et routeur → Agent pharmacie + Agent santé générale → Agent évaluateur → Agent rédacteur	Correct
Comment réparer mon ordinateur ?	Agent lecteur et routeur → Agent santé générale → Agent évaluateur → Agent rédacteur	Correct

Les résultats obtenus montrent que le système respecte le chemin de traitement défini dans l'architecture multi-agents. Les questions médicales sont correctement orientées vers l'agent spécialisé correspondant au domaine détecté, tout en sollicitant l'agent de santé générale afin d'apporter une analyse complémentaire. Les réponses produites sont ensuite vérifiées par l'agent évaluateur avant d'être reformulées et fusionnées par l'agent rédacteur.

Par ailleurs, le test réalisé avec une question hors domaine médical a permis de vérifier le comportement du système face à une requête ne relevant pas du domaine de la santé. Dans ce cas, l'agent routeur classe la requête dans la catégorie générale et la transmet à l'agent de santé générale, qui identifie que la question sort de son domaine d'expertise et

génère une réponse de redirection appropriée. Ce comportement contribue à améliorer la fiabilité du système en limitant la production de réponses inadaptées sur des sujets non médicaux.

3.5.3 Validation du système de surveillance IoT

Cette validation vise à évaluer la capacité du système à analyser les constantes vitales transmises par la montre connectée, à distinguer les situations normales des situations anormales et à générer des alertes adaptées lorsque cela est nécessaire.

Pour réaliser cette validation, plusieurs jeux de données simulant différents états physiologiques ont été envoyés au système. Les résultats obtenus ont ensuite été comparés aux attentes afin de vérifier le bon fonctionnement du processus de surveillance.

Table 3.7 – Validation de la détection des anomalies IoT

Scénario	**Mesures envoyées**	**Résultat attendu**	**Résultat obtenu**
Mesures normales	FC = 78 bpm, SpO2 = 98	Aucune anomalie détectée	Correct
Tachycardie	FC = 150 bpm, SpO2 = 97	Détection d'une tachycardie et génération d'une alerte	Correct
Hypoxie	SpO2 = 85	Détection d'une saturation en oxygène anormale	Correct
Hyperthermie	Température = 40°C	Détection d'une température anormalement élevée	Correct

Les résultats obtenus montrent que le système est capable d'analyser correctement les données physiologiques reçues depuis la montre connectée. Les mesures normales sont enregistrées sans génération d'alerte, tandis que les valeurs dépassant les seuils définis déclenchent automatiquement une analyse approfondie et la génération d'une notification adaptée.

Cette expérimentation confirme ainsi la capacité du système à assurer un suivi automatisé des constantes vitales et à détecter précocement certaines anomalies physiologiques pouvant nécessiter une attention particulière.

3.5.4 Validation de l'apport du mécanisme RAG

Cette validation a pour objectif d'évaluer l'impact du mécanisme de génération augmentée par récupération (RAG) sur la qualité des réponses produites par le système. Pour cela, plusieurs questions ont été soumises au système dans deux configurations différentes :

- **Sans RAG :** les agents s'appuient uniquement sur les connaissances internes du modèle de langage.
- **Avec RAG :** les agents exploitent en complément les ressources médicales présentes dans la base documentaire afin d'enrichir les réponses générées.

L'objectif est de comparer la pertinence, le niveau de détail et la qualité des réponses obtenues dans les deux cas.

3.5.4.1 Comparaison des réponses obtenues

Le tableau 3.8 présente un exemple de comparaison réalisé sur une question relative aux facteurs de maintien de l'anxiété sociale selon la thérapie cognitivo-comportementale.

Table 3.8 – Comparaison des réponses avec et sans RAG

Question	Sans RAG	Avec RAG
Quels sont les facteurs de maintien de l'anxiété sociale selon la thérapie cognitivo-comportementale ?	Réponse correcte présentant les principaux facteurs tels que les croyances dysfonctionnelles, l'évitement des situations sociales, l'hypervigilance et les pensées automatiques négatives.	Réponse plus détaillée et mieux structurée intégrant des concepts complémentaires tels que le conditionnement social, la sensibilité au jugement et le renforcement négatif.

Les figures suivantes illustrent les réponses générées dans les deux configurations.

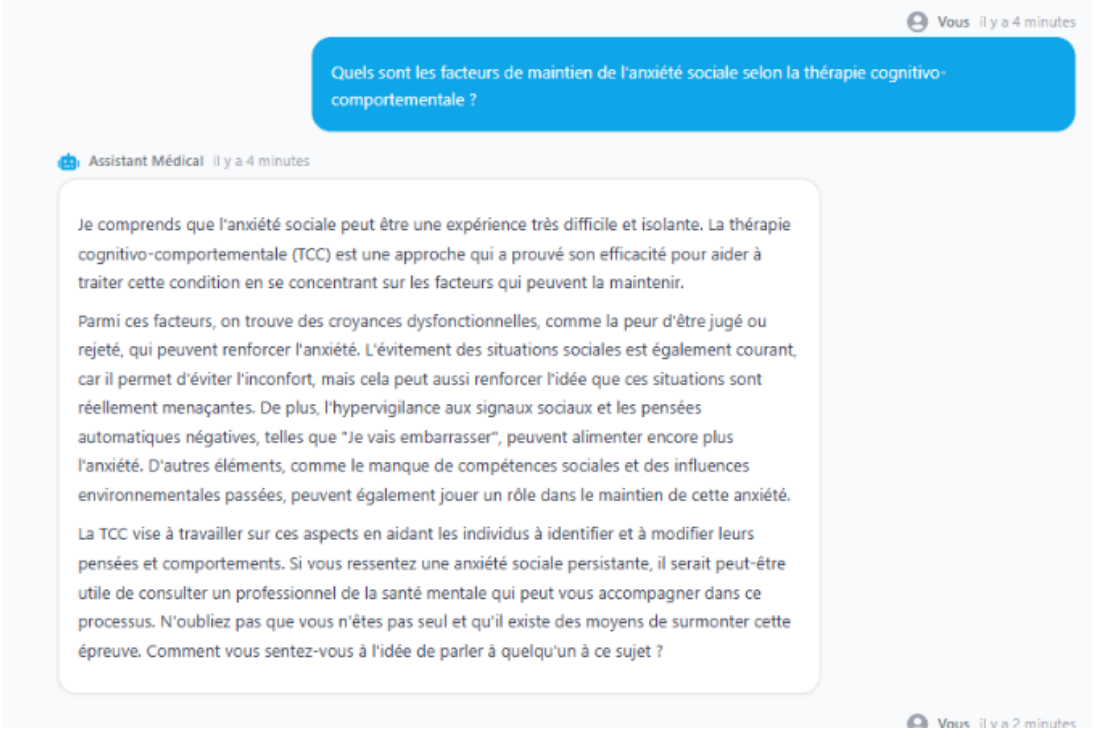


Figure 3.19 – Réponse sans utilisation du mécanisme RAG

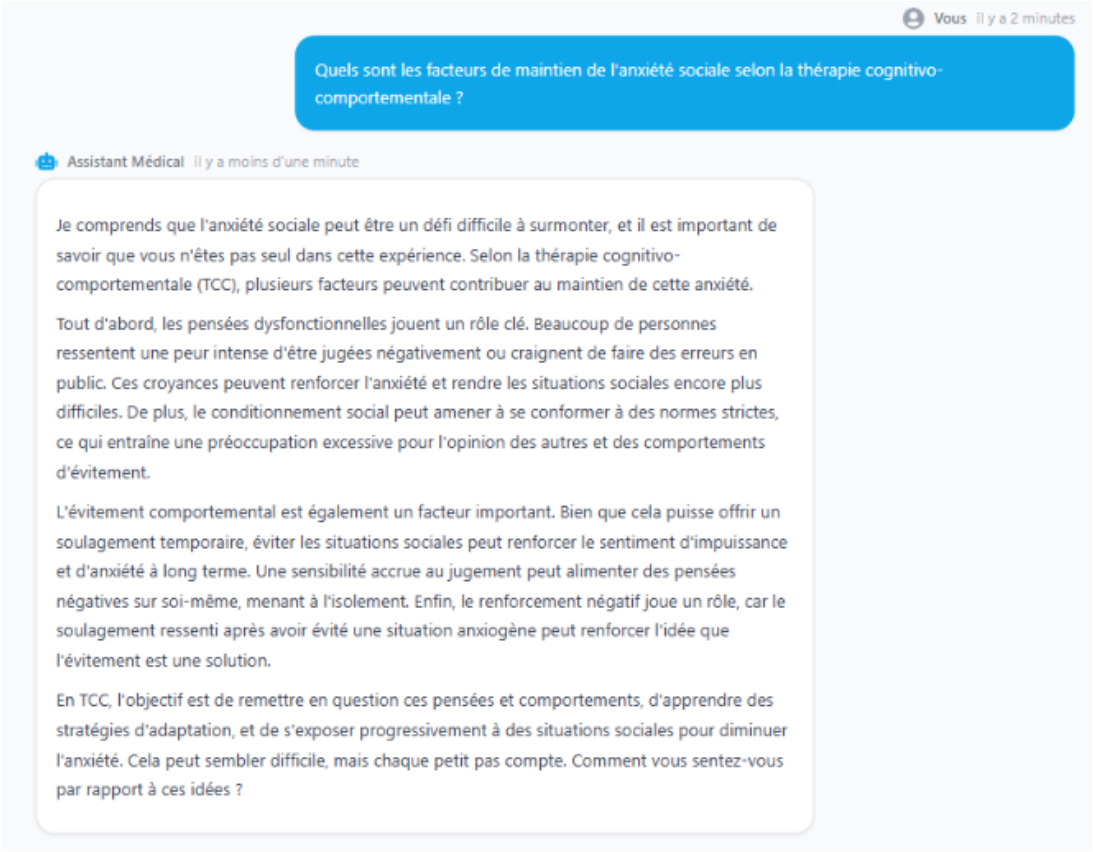


Figure 3.20 – Réponse générée avec utilisation du mécanisme RAG

L'analyse des résultats montre que les deux réponses sont globalement correctes. Cependant, l'utilisation du mécanisme RAG permet d'enrichir davantage le contenu généré grâce aux informations récupérées depuis la base documentaire médicale. Les réponses obtenues sont généralement plus structurées, plus détaillées et présentent des concepts spécialisés qui n'apparaissent pas toujours dans les réponses produites uniquement à partir du modèle de langage.

Cette expérimentation met ainsi en évidence l'intérêt du mécanisme RAG dans notre système. En combinant les capacités de raisonnement du modèle de langage avec une base de connaissances médicale spécialisée, le système est capable de fournir des réponses plus riches, plus contextualisées et potentiellement plus fiables pour l'utilisateur.

Conclusion

Dans ce chapitre, nous avons présenté la réalisation de la plateforme ainsi que les technologies utilisées pour son développement. Les différentes interfaces développées ont permis d'illustrer les principales fonctionnalités du système, notamment l'assistant médical intelligent, la surveillance des constantes vitales et la gestion des alertes médicales. Les résultats obtenus lors des phases de validation confirment le bon fonctionnement de l'architecture multi-agents, l'efficacité du système de surveillance IoT ainsi que l'apport du mécanisme RAG dans l'amélioration de la qualité des réponses générées. Ces résultats démontrent la capacité de la solution proposée à fournir une assistance médicale intelligente et un suivi de santé automatisé.

Conclusion Générale et Perspectives

Ce projet a permis la conception et la réalisation d'un système multi-agents de suivi de santé combinant intelligence artificielle, génération augmentée par récupération (RAG) et technologies de l'Internet des Objets. La solution développée intègre un assistant médical intelligent capable de répondre aux questions des utilisateurs ainsi qu'un système de surveillance exploitant les données physiologiques issues d'une montre connectée.

L'étude menée a permis d'identifier les besoins fonctionnels et non fonctionnels du système, de concevoir une architecture adaptée à l'aide des diagrammes UML et de mettre en œuvre une architecture multi-agents reposant sur plusieurs agents spécialisés collaborant dans le traitement des requêtes médicales. L'intégration du mécanisme RAG et d'une base de connaissances vectorielle a également contribué à améliorer la pertinence et la contextualisation des réponses générées.

Les résultats obtenus lors de la phase de validation montrent que le système est capable de fournir des réponses cohérentes dans différents domaines médicaux, d'assurer le routage correct des requêtes vers les agents spécialisés et de détecter automatiquement certaines anomalies à partir des données physiologiques collectées.

La réalisation de ce projet nous a également permis de consolider nos compétences dans plusieurs domaines, notamment l'intelligence artificielle, les systèmes multi-agents, les modèles de langage, la recherche sémantique, les technologies IoT ainsi que le développement d'applications web modernes.

Plusieurs perspectives d'amélioration peuvent être envisagées. Une première évolution consisterait à enrichir davantage la base de connaissances médicale utilisée par le mécanisme RAG à travers l'intégration de nouvelles sources documentaires et de recommandations médicales récentes. Une seconde perspective concernerait l'ajout de nouveaux agents spécialisés couvrant des domaines médicaux plus spécifiques tels que la cardiologie, la neurologie ou la pédiatrie. Enfin, l'intégration de fonctionnalités multimodales permettant l'analyse de documents médicaux ou d'images de santé pourrait contribuer à étendre les capacités du système et à améliorer la qualité de l'assistance proposée.

Ainsi, ce projet constitue une contribution au développement de solutions intelligentes d'assistance médicale et de suivi de santé, en mettant en évidence l'intérêt de la combinaison entre architectures multi-agents, modèles de langage, mécanismes RAG et technologies connectées.

Références

[1] LangChain Inc., *Retrieval-Augmented Generation (RAG)*. Disponible sur : https://python.langchain.com/docs/concepts/rag/ (consulté en juin 2026).

[2] Jacqueline L. Longe (Ed.), *The Gale Encyclopedia of Medicine*, 5th Edition. Gale, Cengage Learning. Disponible sur : https://www.gale.com (consulté en juin 2026).

[3] Hertfordshire Wellbeing Services, *Cognitive Behavioural Therapy (CBT) Skills Workbook*. Disponible sur : https://www.hpft.nhs.uk/media/1655/wellbeing-team-cbt-workshop-booklet-2016.pdf (consulté en juin 2026).

[4] William J. Knaus, *The Cognitive Behavioral Workbook for Anxiety*. Disponible sur : https://www.friendlysolutionstherapy.com/wp-content/uploads/2023/07/book-6.pdf (consulté en juin 2026).

[5] National Institute of Mental Health (NIMH), *Depression*. Disponible sur : https://www.nimh.nih.gov/sites/default/files/health/publications/depression/depression.pdf (consulté en juin 2026).

[6] Python Software Foundation, *Python Documentation*. Disponible sur : https://docs.python.org/3/ (consulté en juin 2026).

[7] Meta Platforms Inc., *React Documentation*. Disponible sur : https://react.dev/ (consulté en juin 2026).

[8] FastAPI Team, *FastAPI Documentation*. Disponible sur : https://fastapi.tiangolo.com/ (consulté en juin 2026).

[9] CrewAI Inc., *CrewAI Documentation*. Disponible sur : https://docs.crewai.com/ (consulté en juin 2026).

[10] LangChain Inc., *LangChain Documentation*. Disponible sur : https://python.langchain.com/ (consulté en juin 2026).

[11] OpenAI, *OpenAI Models Documentation*. Disponible sur : https://platform.openai.com/docs/models (consulté en juin 2026).

[12] Pinecone Systems Inc., *Pinecone Documentation*. Disponible sur : https://docs.pinecone.io/ (consulté en juin 2026).

[13] SQLite Consortium, *SQLite Documentation*. Disponible sur : https://www.sqlite.org/docs.html (consulté en juin 2026).